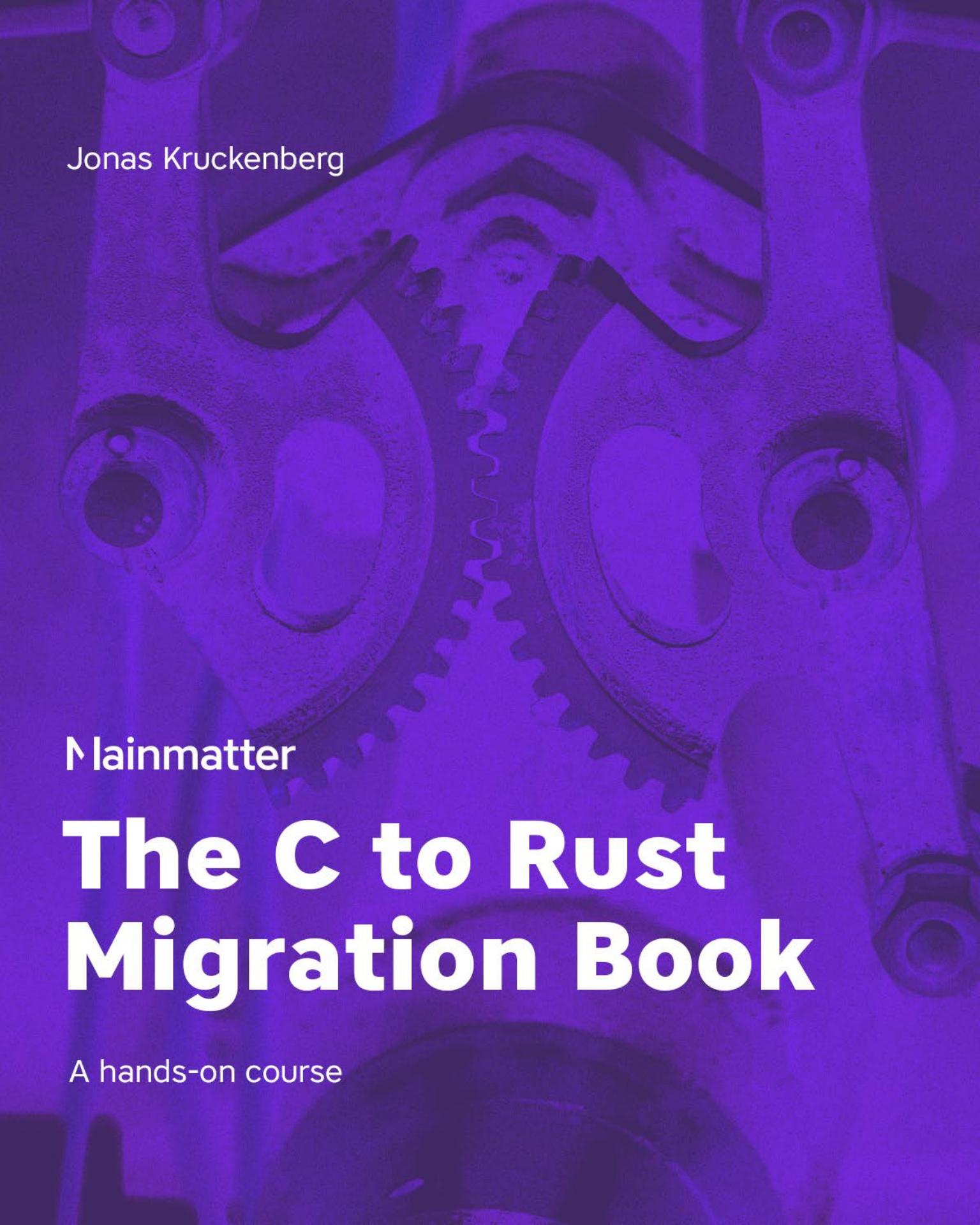


Jonas Kruckenberg

Mainmatter

The C to Rust Migration Book

A hands-on course

The C to Rust Migration Book

A hands-on course by Mainmatter

Jonas Kruckenberg Artur Jakubiec

Contents

Welcome **1**

1 Chapter 1: Unsafe Rust foundations **5**

1.1 `unsafe` and safety contracts . . . 5

1.2 References vs raw pointers . . . 8

1.3 `NonNull<T>` and null-pointer optimization . . . 10

1.4 Memory layout: size, alignment, and `repr` . . . 13

2 Chapter 2: Basics **17**

2.1 What is FFI? . . . 17

2.2 Primitive types in FFI . . . 19

2.3 Structs in FFI . . . 21

2.4 Enums and unions in FFI . . . 23

2.5 Slices in FFI . . . 25

2.6 Opaque types . . . 27

2.7 When hand-written bindings drift . . . 31

2.8 Generating Rust bindings with `bindgen` . . . 33

2.9 Exposing Rust to C with `cheadergen` . . . 34

2.10 The FFI boundary as a firewall: validate and narrow . . . 36

2.11 High-quality FFI APIs . . . 38

3 Chapter 3: Intermediate **41**

3.1 How to rewrite a module . . . 41

3.2 Allocation, allocators, and mixing them . . . 44

3.3 Using `lldb/gdb` . . . 47

3.4 Clippy lints for unsafe Rust and static verification . . . 50

3.5 Out-parameters . . . 53

3.6 Error codes 55
3.7 Strings 58
3.8 Bit flags 60
3.9 Loops and iterators 62
3.10 Dynamic analysis with Miri 64
3.11 Performance benchmarking 67

Welcome

Welcome to the **"C to Rust Migration Book"**!

This course will teach you techniques for migrating C codebases to Rust.
You'll learn how to maintain a safe mixed C-Rust codebase, incrementally migrate modules, translate common C idioms to Rust, and use the debugging tools you'll need when things inevitably break.

We assume prior knowledge of Rust and some C.

You'll build up your knowledge in small, manageable steps. By the end of the course, you will have solved many exercises and should be prepared to migrate even larger C codebases to Rust.

Why FFI matters

Rewriting a large C codebase in one go rarely works. Instead, you migrate it piece by piece, and for a while C and Rust live in the same program and call each other. What makes that possible is a *Foreign Function Interface* (FFI): a way for code in one language to call functions and use types defined in another. Nearly every language supports the same one: calling functions as if they were written in C.

The boundary between the two languages is where most of the difficulty lives. Rust's compiler can't check the C side, and C doesn't know about Rust's rules, so everything that crosses has to be described, validated, and owned correctly by hand. This course starts with the `unsafe` Rust you need to reason about that boundary, then covers how values cross it, and finally how to move code from one side to the other.

Methodology

This course is based on the "learn by doing" principle.
It has been designed to be interactive and hands-on.

Mainmatter developed this course to be delivered in a classroom setting, over four days: each attendee advances through the lessons at their own pace, with an experienced instructor providing guidance, answering questions, and diving deeper into the topics as needed.
If you'd like to organize a private session for your company, please get in touch.

You can also take the course on your own, but we recommend you find a friend or a mentor to help you along the way should you get stuck.

Formats

You can go through the course material [in the browser]([TODO: insert URL]) or [download it as a PDF file]([TODO: insert URL]), for offline reading.

Structure

On the left side of the screen, you can see that the course is divided into sections. To verify your understanding, each section is paired with an exercise that you need to solve.

You can find the exercises in the companion GitHub repository.
Before starting the course, make sure to clone the repository to your local machine:

```
git clone https://github.com/mainmatter/migrating-c-to-rust
```

We also recommend you work on a branch, so you can easily track your progress and pull in updates from the main repository, if needed:

```
cd migrating-c-to-rust
git checkout -b my-solutions
```

All exercises are located in the `exercises` folder. Each exercise is structured as a Rust package. The package contains the exercise itself, instructions on what to do (in `src/lib.rs`), and a test suite to automatically verify your solution.

Tools

To work through this course, you'll need:

- **Rust**, installed through `rustup`. The repository pins a nightly toolchain in `rust-toolchain.toml`, because some of the tools the course uses, such as Miri, only run on nightly. `rustup` picks it up whenever you run `cargo` inside the repository. If it isn't installed yet, run `rustup toolchain install` from the repository root.
- *(Optional but recommended)* An IDE with Rust autocompletion support. We recommend one of the following:
  - RustRover;
  - Visual Studio Code with the `rust-analyzer` extension.
- A C compiler. The one provided by your operating system will be good enough.
- Two extra compilation targets. One exercise type-checks your bindings against platforms that disagree with yours, which needs their standard libraries. Run this from the repository root, so the targets are added to the pinned toolchain:

```
rustup target add x86_64-unknown-linux-gnu aarch64-unknown-linux-gnu
```

Nothing is compiled or linked for those targets, so you don't need a cross compiler.

Workshop runner

To verify your solutions, we've also provided a tool to guide you through the course: the `wr` CLI, short for "workshop runner". Install `wr` by following the instructions on its website.

Once you have `wr` installed, open a new terminal and navigate to the top-level folder of the repository. Run the `wr` command to start the course:

```
wr
```

`wr` will verify the solution to the current exercise.
Don't move on to the next section until you've solved the exercise for the current one.

> We recommend committing your solutions to Git as you progress through the course, so you can easily track your progress and "restart" from a known point if needed.

Enjoy the course!

Authors

This course was written by Jonas Kruckenberg, Engineering Consultant at Mainmatter, and Artur Jakubiec.
Jonas Kruckenberg is a systems engineer working on operating systems and low-level infrastructure. He is the lead author of k23, an experimental high-reliability operating system. As a TC39 Invited Expert, he helps shape the future of the web by bringing non-browser perspectives to JavaScript language standardization.
Artur Jakubiec is a long-time C/C++ engineer and was one of the main developers of Cubasis. He is now a tech lead at a company building enterprise SaaS productivity software, currently migrating it from C++ to Rust.

Mainmatter is an engineering consultancy specializing in Rust. We offer engineering services, training, and mentorship to companies that aim to migrate from C to Rust or introduce Rust into an existing codebase.

License

Copyright © 2024- Mainmatter GmbH (https://mainmatter.com), released under the PolyForm-Noncommercial-1.0.0.

Chapter 1

Chapter 1: Unsafe Rust foundations

Safe Rust comes with a guarantee: if the program compiles, it has no undefined behavior. The compiler enforces the rules for references, valid values, and mutation that make that guarantee possible. Those checks stop at the language boundary, though. Rust cannot borrow-check C, and C does not enforce Rust's invariants. It falls to you to know what each language guarantees about memory, what it assumes, and what happens when those assumptions don't hold.

This chapter covers the building blocks: what `unsafe` means and what it does and doesn't change, how raw pointers differ from references, how `NonNull` and `Option` let the type system track null pointers, and how size, alignment, and padding work.

Exercises

The exercises for this chapter are located in `exercises/01_foundations`.

Exercise

The exercise for this section is located in `01_foundations/00_README` \newpage{=latex}

1.1 `unsafe` and safety contracts

Safe Rust can't cause undefined behavior, but some things are beyond what the compiler can check, such as whether a pointer handed over by a C library still points to live memory. For those, Rust has the `unsafe` keyword. It marks the places where you, instead of the compiler, are responsible for keeping the program sound.

What `unsafe` unlocks

`unsafe` doesn't switch off the borrow checker or any other check. It lets you perform a small set of extra operations the compiler can't verify:

- dereferencing a raw pointer
- calling an `unsafe` function, which includes most functions implemented in C
- reading or writing a `static mut` or an `extern static`
- reading a field of a `union`
- implementing an `unsafe` trait such as `Send` or `Sync`

Everything else inside an `unsafe` block is checked exactly as it would be outside of one.

Undefined behavior

The rules these operations can break all lead to *undefined behavior* (UB). If a program has UB, Rust no longer defines what it does: it might crash, give wrong results, or seem to work fine. The compiler assumes UB never happens and optimizes based on that, so a program that seems fine today can break with the next compiler version.

The Rust Reference keeps the full list of what counts as UB. The entries you'll meet most often in FFI code are:

- dereferencing a null, dangling, or misaligned pointer
- producing an invalid value, such as a `bool` holding 3 or an enum with a discriminant that doesn't match any variant
- breaking the aliasing rules, for example mutating memory while a shared reference to it exists
- reading uninitialized memory
- data races

Safety contracts

An `unsafe` operation always comes with conditions that make it sound. We call those conditions the operation's *safety contract*. Rust splits the contract into two halves.

An `unsafe fn` declaration says "calling me is only sound if you, the caller, uphold these conditions". The conditions go into a `# Safety` section of the doc comment:

```
/// Reads the `u32` that `ptr` points to.
///
/// # Safety
///
/// `ptr` must be non-null, aligned for `u32`, and point to an initialized `u32`
/// that stays alive and remains unmodified for the duration of the call.
pub unsafe fn read_u32(ptr: *const u32) -> u32 {
    // SAFETY: the caller guarantees that `ptr` points to a valid `u32`.
    unsafe { *ptr }
}
```

An `unsafe` block says "I checked the conditions here". The justification goes into a `// SAFETY:` comment right above it:

```
/// # Safety
/// `ptr` must be non-null, aligned, and point to an initialized `u32`.
unsafe fn read_u32(ptr: *const u32) -> u32 { unsafe { *ptr } }
let x = 41_u32;
// SAFETY: `&x` is non-null and aligned, and `x` is an initialized `u32` that
// outlives the call.
let y = unsafe { read_u32(&x) };
```

A `// SAFETY:` comment explains why the code upholds each condition, or, inside an `unsafe fn`, points to the condition its caller already promised to uphold.

Safe abstractions and soundness

It's common for `unsafe` code to end up wrapped in a safe function. The safe function checks whatever the contract requires, then performs the `unsafe` operation. That wrapper is only correct if no input, however strange, can cause UB. A safe function that can cause UB is called *unsound*, and it's a bug even if no caller triggers it today.

Here's an unsound one:

```
/// Returns the first byte of `bytes`.
pub fn first(bytes: &[u8]) -> u8 {
    // SAFETY: ??? An empty slice has no index 0.
    unsafe { *bytes.get_unchecked(0) }
}
```

Calling `first(&[])` reads out of bounds, and nothing in the signature warns the caller about it. The sound version checks the condition before relying on it:

```
/// Returns the first byte of `bytes`, or `None` if `bytes` is empty.
pub fn first(bytes: &[u8]) -> Option<u8> {
    if bytes.is_empty() {
        return None;
    }
    // SAFETY: `bytes` is not empty, so index 0 is in bounds.
    Some(unsafe { *bytes.get_unchecked(0) })
}
```

In real code you would write `bytes.first().copied()`, which does the same check without any `unsafe`. The example uses `get_unchecked` only to show the pattern: check, then justify, then perform the operation. Writing the `// SAFETY:` comment is often how you find out that a check is missing. If you can't finish the sentence, the code isn't sound yet.

Head to the exercise

The exercise is one function: return the last byte of a slice, or `None` if the slice is empty, reading the byte with `get_unchecked`. Work out the condition that makes the index valid before you rely on it, and say why in a `// SAFETY:` comment.

Exercise

The exercise for this section is located in `01_foundations/01_unsafe` \newpage{=latex}

1.2 References vs raw pointers

C has one kind of pointer. Whether a `T *` may be null, who owns the memory behind it, and how long it stays valid are all conventions, written down in documentation if you're lucky. Rust has references instead, and a reference carries those guarantees in its type.

What a reference guarantees

A reference, `&T` or `&mut T`, is always:

- **non-null**
- **aligned** for `T`
- pointing to a **live, initialized, valid** `T`
- valid for its **lifetime**, which the borrow checker enforces
- subject to the **aliasing rules**: either any number of `&T`, or exactly one `&mut T`

These guarantees aren't only there to protect you. The compiler optimizes based on them. Because a `&mut T` is the only way to reach its value, the compiler can keep that value in a register instead of re-reading it from memory.

That's also why creating a reference that breaks one of these guarantees is undefined behavior, even if you never read through it.

Raw pointers

Those guarantees can be quite limiting, though, especially when you want to interface with C. For this reason Rust also has raw pointers: `*const T` and `*mut T`. They are Rust's equivalent of C's `const T *` and `T *`, and they come without any of the guarantees Rust has for references. A raw pointer may be null, dangling, misaligned, or pointing to memory that is being mutated through another pointer at the same time.

Creating a raw pointer is a safe operation. Dereferencing one is `unsafe`, because that's the moment all those possibilities matter:

```
let mut x = 10_i32;

// Creating raw pointers is safe.
let p: *mut i32 = &mut x;
let null: *const i32 = std::ptr::null();
assert!(!p.is_null());
assert!(null.is_null());

// SAFETY: `p` comes from `&mut x`, so it's non-null, aligned, and points to a
```

```
// live `i32`. Nothing else accesses `x` while we use `p`.
unsafe {
    *p += 1;
    assert_eq!(p.read(), 11);
}
```

Raw pointers have a small API of their own. The methods you'll mostly use are:

- `is_null` to check for null
- `cast` to change the pointee type, the equivalent of a C pointer cast
- add for pointer arithmetic, which is `unsafe` because the result must stay within the same allocation
- `read` and `write` to copy a value out or in without creating a reference

Mutability is about where a pointer came from

In C, casting away `const` is legal as long as the memory itself isn't `const`. Rust is stricter. Whether you may write through a pointer depends on how it was created, not only on its type. A `*mut T` that you got by casting a `&T` must never be written through, because a `&T` guarantees that the value doesn't change.

The compiler catches the most obvious version of this:

```
let x = 10_i32;
let p = &x as *const i32 as *mut i32;
// error: assigning to `&T` is undefined behavior, consider using an `UnsafeCell`
unsafe { *p = 20 };
```

It doesn't catch the less obvious ones. This example compiles, runs, and prints `20`, but it is undefined behavior all the same:

```
let mut x = 10_i32;
let p = &mut x as *mut i32;
let r = &x;
// SAFETY: none! `r` is a shared reference, so `x` must not change while it
// is alive.
unsafe { *p = 20 };
println!("{r}");
```

The borrow checker only tracks references, so it has no idea `p` and `r` point to the same value. Miri does catch it and reports undefined behavior; Dynamic analysis with Miri puts it to work on a real port.

Keep in mind that C's `const` is a convention. A C function that takes a `const T *` can still cast it away and write through it. If you pass it a pointer derived from a `&T`, you are trusting that it doesn't.

From raw pointer to reference

Sooner or later you'll want to turn a raw pointer from C into a reference, so the rest of your Rust code can work with it safely. There are two ways to do that:

- `&*ptr` or `&mut *ptr` dereference the pointer and borrow the result
- `ptr.as_ref()` and `ptr.as_mut()` do the same, but return None for a null pointer

Both approaches are `unsafe`. The compiler can't check that the pointer meets the guarantees a reference needs, so you have to promise that it does. It also can't work out how long the memory stays valid, so you have to assign a lifetime for the reference that matches how long the C side keeps the value alive.

```
pub struct Config {
    pub verbose: bool,
}

/// Borrows the configuration a C library handed us.
///
/// # Safety
///
/// `ptr` must be null, or point to a valid `Config` that stays alive and
/// unmodified for as long as the returned reference is in use.
pub unsafe fn config_from_c<'a>(ptr: *const Config) -> Option<&'a Config> {
    // SAFETY: the caller guarantees that `ptr` is null or valid for `'a`.
    unsafe { ptr.as_ref() }
}
```

The `# Safety` section tells you what `'a` means in practice. Without it, a caller could keep the reference around long after the C library freed the `Config`, and nothing would stop them.

Head to the exercise

The exercise is one function: swap the two `i32`s that a pair of raw pointers point to.

Exercise

The exercise for this section is located in `01_foundations/02_raw_pointers` \newpage{=latex}

1.3 `NonNull<T>` and null-pointer optimization

In FFI code you often know that a pointer isn't null. `NonNull<T>` puts that knowledge in the type. Wrapped in an `Option`, it also gives you a pointer that may be null, but that you can't use without checking first.

NonNull<T>

NonNull<T> behaves like a *mut T with one extra guarantee: it is never null. You create one from a raw pointer with NonNull::new, which returns None for null, or from a reference, which is never null to begin with:

```
use std::ptr::NonNull;

let mut x = 5_u32;
let p: NonNull<u32> = NonNull::from(&mut x);

// SAFETY: `p` comes from `&mut x`, which is alive and not borrowed elsewhere.
unsafe { *p.as_ptr() += 1 };
assert_eq!(x, 6);

let from_c: *mut u32 = std::ptr::null_mut();
assert!(NonNull::new(from_c).is_none());
```

A few more methods are worth knowing:

- NonNull::new_unchecked skips the null check. It's unsafe, because passing a null pointer is undefined behavior.
- as_ptr turns it back into a *mut T, for example to hand it to C.
- as_ref and as_mut turn it into a reference. They're unsafe for the same reasons as with raw pointers: non-null doesn't mean aligned, alive, or valid.
- NonNull::dangling creates a non-null, aligned pointer that doesn't point to anything. The standard library uses it for empty collections, which have nothing to point at but still have to hold a pointer.[1]

Null-pointer optimization

A NonNull<T> can never be all zeros (0 being a forbidden value is literally the type's only job), and the compiler takes advantage of that. A bit pattern that a type can never hold is called a *niche*, and the compiler looks for one whenever it lays out an enum. For pointer-like types the trick has a name of its own: *null-pointer optimization* (NPO).

```
use std::mem::size_of;
use std::ptr::NonNull;

const _: () = {
    assert!(size_of::<Option<NonNull<u8>>>() == size_of::<NonNull<u8>>());
    assert!(size_of::<Option<&u8>>() == size_of::<&u8>());
    assert!(size_of::<Option<Box<u8>>>() == size_of::<Box<u8>>());
    assert!(size_of::<Option<extern "C" fn()>>() == size_of::<extern "C" fn()>());
};
```

A raw pointer has none of these guarantees. Every combination of bits can be a valid *const T or *mut T, so there's no niche left to mark None with. The compiler has to store that separately, and pad the result back out to the pointer's alignment. On a 64-bit target that doubles the size:

[1] Vec and Box don't store a NonNull<T> directly. They use an internal Unique<T>, which wraps NonNull<T> and adds the claim that the pointer is the only one to its value, the way a &mut T is.

```rust
use std::mem::size_of;

const _: () = {
    assert!(size_of::<*const u8>() == 8);
    assert!(size_of::<Option<*const u8>>() == 16);
};
```

That size is a symptom rather than the rule: `Option<*const T>` has no specified layout, so it wouldn't be a C pointer even if it happened to be eight bytes wide. That is the practical reason `NonNull` exists: `Option<NonNull<T>>` lets you say "may be null" in Rust's type system and get a value that is one pointer wide, with a layout you're allowed to rely on.

What's guaranteed

Niches are an optimization, and in general the compiler is free to decide how to lay out an `Option`. For a short list of types, though, the standard library guarantees that `Option<T>` has the same size, alignment, and function-call ABI as `T`, with None represented as all zeros:

- `&T` and `&mut T`
- `Box<T>`
- `NonNull<T>`
- `NonZero<u8>`, `NonZero<u32>`, and the other `NonZero` integers
- function pointers, including `extern "C" fn`
- `#[repr(transparent)]` structs around one of the above

For the pointer types, the all-zeros part only holds when `T` is sized. A pointer to an unsized type, such as `&[u8]` or `&str`, is a pointer plus a length, and can't cross the FFI boundary as a single C pointer.

This guarantee is what makes these types usable in FFI signatures. For anything else, such as `Option<bool>` or `Option<u32>`, the layout is a compiler decision and can't be used in FFI signatures.

Head to the exercise

The exercise is one function: take a raw pointer the way C would hand it over, read a field out of what it points to, and return None when there is nothing to read. Let the type carry the null check rather than writing one by hand.

Exercise

The exercise for this section is located in `01_foundations/03_nonnull` \newpage{=latex}

1.4 Memory layout: size, alignment, and `repr`

When C and Rust share data, both sides have to agree on its layout: how many bytes a value takes, and where each field sits inside them. C types derive their default layout rules from the platform ABI. Rust types, by default, have an unspecified layout optimized by the compiler. This section explains how to ensure that size and alignment between C and Rust match predictably across language boundaries.

Size and alignment

Every type has a size and an alignment. The *size* is how many bytes a value takes up. The *alignment* is a number that the address of a value has to be a multiple of. Both are known at compile time:

```
assert_eq!((size_of::<u8>(), align_of::<u8>()), (1, 1));
assert_eq!((size_of::<u32>(), align_of::<u32>()), (4, 4));
assert_eq!((size_of::<[u32; 3]>(), align_of::<[u32; 3]>()), (12, 4));
```

For primitives, alignment usually equals size. Alignment comes from the hardware: misaligned loads and stores can be much slower than aligned accesses, sometimes requiring the compiler to emit multiple instructions; on some architectures they even raise a fault. Rust doesn't distinguish between architectures here: accessing a value through a misaligned pointer is undefined behavior on every target.[2]

Layout

The two numbers travel together often enough that the standard library has a type for the pair. `std::alloc::Layout` carries a size and an alignment, describing either a single value or an array of them:

```
use std::alloc::Layout;

let one = Layout::new::<u64>();
assert_eq!((one.size(), one.align()), (8, 8));

let three = Layout::array::<u64>(3).unwrap();
assert_eq!((three.size(), three.align()), (24, 8));
```

`Layout::array` returns a `Result`, because the total size of a large array can overflow.

Allocation APIs are written in terms of `Layout`: an allocator has to know how many bytes to allocate and how they need to be aligned. Allocation, allocators, and mixing them covers the allocator API in more detail.

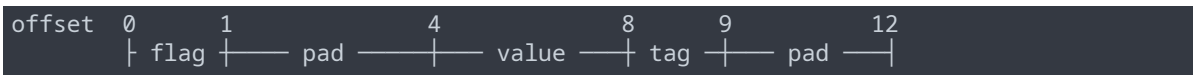
[2] When the data really is misaligned, such as a struct read straight out of a network packet, reach for `ptr::read_unaligned` and its `write_unaligned` counterpart. They copy the bytes without ever forming a misaligned reference, at the cost of the slower access the hardware needs for it.

How a struct gets its size and alignment

In C, a struct's size and alignment are calculated from its fields. The fields are placed one after the other, but each one has to start at an offset that is a multiple of its own alignment, so a field may start a few bytes after the previous one ends. The struct as a whole gets the alignment of its most-aligned field, and its size is rounded up to a multiple of that alignment, so that the next value in an array starts aligned too.

Here is a C struct and its layout:

```c
struct sample {
    bool flag;
    uint32_t value;
    uint8_t tag;
};
```



`flag` sits at offset 0. `value` needs an offset that is a multiple of 4, so it starts at 4. `tag` follows at 8. The struct's alignment is 4, from `value`, so its size is rounded up to 12.

The unused bytes the compiler inserts to keep fields aligned are called *padding*: the three after `flag` and the three after `tag`. That's 6 bytes of data in 12 bytes of memory.

No guarantees by default

Rust gives the equivalent struct no specified layout:

```rust
struct Sample {
    flag: bool,
    value: u32,
    tag: u8,
}
```

The compiler may place these three fields in any order. With `value` first, the same data requires less padding and fits in 8 bytes instead of 12.

For Rust-only code that is a good trade, because the compiler picks a tighter layout than a fixed order would and nothing outside the program depends on the result. It stops working as soon as C is involved: the C side computes field offsets from the struct definition, and those offsets follow C's rules rather than whatever the Rust compiler decided.

What `repr` is for

`repr` is an attribute on a type definition that replaces the default layout with a specified one. There are two reasons to reach for it:

- **The layout has to match a definition outside the program.** A struct C reads or writes, a value passed to a syscall, a header from a file format or a network packet. Something else already fixed the byte positions, and the type has to reproduce them.
- **The padding or alignment itself matters.** Removing padding to match a packed wire format, or raising alignment because hardware asks for it.

`repr` applies to `struct`, `enum`, and `union` definitions, and it describes the whole type: there is no way to give an individual field its own representation, and Rust has no equivalent of C's bitfields.

The variant you'll use most is `repr(C)`, which lays a type out by C's rules. The others cover enum discriminants, single-field wrappers, and changes to padding and alignment, and each one comes up together with the problem it solves.

Head to the exercise

The exercise has two halves. First, work out the size and alignment of a few types on paper and write the numbers down as constants, which the tests compare against what the compiler reports. Then take a `repr(C)` struct that carries 14 bytes of fields in 24 bytes of memory, and reorder its fields so it carries them in 16.

Exercise

The exercise for this section is located in `01_foundations/04_layout` \newpage{=latex}

Chapter 2

Chapter 2: Basics

In this chapter we'll cover the FFI foundations: what FFI is, how we use it to interoperate between C and Rust, and the common pitfalls.

We start with the mechanics of declaring and exporting functions, then work through the kinds of data that cross the boundary: primitive types, structs, enums and unions, slices, and opaque handles. Next we look at why hand-written bindings drift out of sync with the C they describe, and at the tools that generate them for you: `bindgen` for Rust bindings to C, and `cheadergen` for C headers from Rust. The chapter ends with the rules we follow to make an FFI boundary trustworthy: validating everything that comes in, and designing an API that is small and documented well enough to keep correct.

Exercises

The exercises for this chapter are located in `exercises/02_basics`.

Exercise

The exercise for this section is located in `02_basics/00_README` `\newpage{=latex}`

2.1 What is FFI?

A *Foreign Function Interface* (FFI) bridges different type systems (or lack of type systems), execution models, and code-organization concepts, so that code in one language can call functions and use types from another. The exact syntax and options vary by language, but they almost always have one thing in common: they represent functions as if they were C functions.

This is called the "C ABI" (C Application Binary Interface) or "C calling convention" and, as the name suggests, is a convention on how to call functions (which CPU registers hold what argu-

ments, which register(s) hold return values, when to spill to the stack, etc.)[1]. Notice that this is just a convention that the industry agreed on over time. The C ABI is predictable, simple, and stable, and it has been so for decades, which is how it became the de facto standard for language interoperability.

Calling an FFI function

Let's say for example we have a Rust program that needs to call the `time` function from `libc` (a C static library)[2]. We would use the following construct:

```
type time_t = i64;
unsafe extern "C" {
    fn time(time: *mut time_t) -> time_t;
}
```

Before we dissect the syntax though: you may have already noticed that nowhere in this snippet do we ask for `libc`. Why is that?

This is because object file formats (ELF on Linux, Mach-O on macOS, PE on Windows) are all old and therefore simplistic. They have *one global namespace*, called a *symbol table*, that all functions (and statics) share. So you cannot say "call `time` from `libc`", you can only say "call a function named `time`".

When a compiler builds a program, all function calls reference the function *by symbol* ("call function named X"). To make this actually executable, we need to replace every symbol reference with the actual address of the function. This happens *after* compilation during the *linking* step, where a separate program, the *linker*, aggregates all object files that make up your final program, lays them out on disk, and then resolves these symbol names.

To call `time` from `libc` we therefore need Rust to emit a reference to the `time` symbol and make sure the `libc` file is also passed to the linker.

This solves the problem of *what* to call, but we still need to figure out *how* to call that function: How many parameters does the function accept? What are the types of those parameters? How many values does it return? Remember the symbol references above are plain string names. They carry no information about the function's argument or return types, so `rustc` has no way to work them out on its own.

This is why we need to tell it about `time`'s signature through a so-called "extern block" (or sometimes an "extern C block") above. This block *declares* items that are not *defined* in the current crate. Each item we declare is a promise to the compiler: "this is the correct signature of this symbol, trust me". We commonly refer to it as a **binding**.

Get the signature wrong and your Rust program will pass garbage to the FFI function without any way to check this at compile-time. The exact implementation won't be known until link-time,

[1] Technically, there is no "single" calling convention. Every architecture defines its own "C Calling Convention". For example, the RISC-V C Calling Convention is defined here and lays out the sizes of C primitives and how arguments and return values are passed from and to functions. x86 architectures have *many* different calling conventions: the Microsoft x64 calling convention and the System V ABI are the most common, but many calling conventions exist to improve calling performance (`fastcall`, `regcall`, and more). When we say "the C calling convention" we usually mean "the C calling convention commonly used on this OS+architecture combination".

[2] Yes, generally speaking `libc` is distributed not as a static but as a *dynamic* library, which is a completely different way of linking and calling functions.

much later than the compiler's type-checking pass. This is why bindings are marked `unsafe`: you as the programmer have to ensure signatures are correct.

This is the foundation of all Foreign Function Interfaces in Rust. In later exercises we will see how to make this much safer and more ergonomic.

Calling Rust from C

A migration also needs the other direction: C code calling functions that are now implemented in Rust. That takes two annotations on an ordinary Rust function:

```
#[unsafe(no_mangle)]
pub extern "C" fn add(a: i32, b: i32) -> i32 {
    a + b
}
```

`extern "C" fn` tells `rustc` to use the C calling convention for this function. `#[unsafe(no_mangle)]` keeps the symbol name exactly as written.

Head to the exercise

You'll write this block for a `bm_add` function implemented in C.

Exercise

The exercise for this section is located in `02_basics/01_what_is_ffi` \newpage{=latex}

2.2 Primitive types in FFI

So far, the exercises have passed data between C and Rust mostly as pointers and plain integers. The following sections cover how other types cross the FFI boundary, starting with primitive types, then structs, enums and unions, slices, and opaque types.

A C header describes its parameters in C's own types: `int`, `long`, `char`, `size_t`. Rust has `i32`, `i64`, `u8`, `usize`. Writing a binding means pairing them up, and most of those pairs work on every platform you're likely to target.

Take this function:

```
int scale(int value, int factor);
```

On Windows, macOS, and Linux, on x86-64 and aarch64, an `int` is the same as an `i32` in Rust. If you only target those platforms, the Rust code looks like this:

```
unsafe extern "C" {
    fn scale(value: i32, factor: i32) -> i32;
}
```

The mapping

Here is how the common primitive types map from C to Rust when targeting Windows, macOS, or Linux on x86-64 or aarch64:

C type	Rust type	Alias
`char`	`i8` (or `u8` on aarch64 Linux)	`c_char`
`signed char`	`i8`	`c_schar`
`unsigned char`	`u8`	`c_uchar`
`short`	`i16`	`c_short`
`unsigned short`	`u16`	`c_ushort`
`int`	`i32`	`c_int`
`unsigned int`	`u32`	`c_uint`
`long`	`i64` (or `i32` on Windows)	`c_long`
`unsigned long`	`u64` (or `u32` on Windows)	`c_ulong`
`long long`	`i64`	`c_longlong`
`unsigned long long`	`u64`	`c_ulonglong`
`float`	`f32`	`c_float`
`double`	`f64`	`c_double`
`size_t`	`usize`	–
`ptrdiff_t`	`isize`	–
`int32_t`, `uint64_t`, …	`i32`, `u64`, …	–
`void *`	`*mut c_void`	–
`T *`	`*mut T`	–
`const T *`	`*const T`	–

The names in the last column are Rust type aliases for each C type. Each alias stands for whichever Rust type matches that C type on the platform you compile for.

They are defined in `core::ffi`, and `std::ffi` re-exports them, so `std::ffi::c_int` and `core::ffi::c_int` are the same type. You'll also see `std::os::raw::c_int` in bindgen output and `libc::c_int` from the `libc` crate. All of them resolve to the same alias.

The aliases are for code that targets platforms where a fixed-width Rust type doesn't match. On MSP430 or AVR, for example, C's `int` is 16 bits, so `c_int` is `i16`. If your code has to build there as well as on x86-64, where `c_int` is an `i32`, the alias is the right option.

`c_char` and `u8`

A C `char` is a byte, and Rust's byte type is `u8`, so writing `*const u8` in a binding where the header says `const char *` looks harmless, and it's the most common mistake in hand-written bindings: `c_char` is not always `u8`. It is `i8` on x86-64, and on macOS and Windows for aarch64. It is `u8` on aarch64 Linux, on 32-bit ARM, and on RISC-V.

The u8 binding might just work on your own machine, which is what makes this easy to miss. Then someone builds for a target where `c_char` is an `i8` and the code stops compiling. Anything that hands you a `c_char` no longer lines up with the u8 you wrote: `CStr::as_ptr` gives a `*const c_char`, and passing it where a `*const u8` is expected is now a type mismatch. So write `c_char` in the signature.

The second issue is that `c_char` is not the type Rust's byte methods work on. `b'a'` literals, `u8::to_ascii_lowercase`, `str::as_bytes`, and `CStr::to_bytes` all speak u8. So cast a `c_char` to u8 as soon as you have the value, do the work there, and cast back on the way out:

```
use std::ffi::c_char;

/// Lowercases `byte` if it is an ASCII letter, and returns it unchanged
/// otherwise.
#[unsafe(no_mangle)]
pub extern "C" fn lower_byte(byte: c_char) -> c_char {
    (byte as u8).to_ascii_lowercase() as c_char
}
```

Two details about those casts are worth naming:

- **They change the type, not the value.** `as` between `c_char` and u8 keeps the bit pattern exactly, and casting back gives you the same byte again.
- **They are allowed because the layouts match.** `c_char` and u8 have the same size and the same alignment on every target, and every bit pattern is valid for both. That makes the cast purely a compile time check.

Head to the exercise

The exercise gives you a C library over bytes with its bindings already written, and two small wrappers to finish.

Exercise

The exercise for this section is located in `02_basics/02_primitives` \newpage{=latex}

2.3 Structs in FFI

Memory layout showed that a Rust struct has no specified layout unless you ask for one, and that `repr(C)` asks for the one a C compiler would pick. This section covers a few more things to know about structs in FFI.

What can go in the fields

A struct can only cross the FFI boundary if all of its fields can. These types are safe to use as fields:

- **Primitives and raw pointers**, mapped as in Primitive types in FFI.
- **Other `repr(C)` structs**, nested by value, as well as enums and unions with a C-compatible `repr`, which the next section covers.
- **Fixed-size arrays.** A C field `char name[16]` is `[c_char; 16]` in Rust, stored inline in the struct, exactly as in C.
- **Nullable pointers as an `Option`.** `Option<NonNull<T>>`, `Option<&T>`, and `Option<extern "C" fn(...)>` are guaranteed to be one pointer wide, with `None` as null.

Types whose layout Rust keeps to itself, such as `String`, `Vec<T>`, `Box<dyn Trait>`, tuples, and slices, can't go in a shared struct. The `improper_ctypes` lint warns when a struct used in an `extern` block lacks `repr(C)` or has a field that can't cross.

By value or by pointer

A C function can take or return a struct by value, or work through a pointer to one:

```c
struct Point {
    double x;
    double y;
};

struct Point midpoint(struct Point a, struct Point b);
void translate(struct Point *point, double dx, double dy);
```

```rust
#[repr(C)]
#[derive(Clone, Copy)]
pub struct Point {
    pub x: f64,
    pub y: f64,
}

unsafe extern "C" {
    fn midpoint(a: Point, b: Point) -> Point;
    fn translate(point: *mut Point, dx: f64, dy: f64);
}
```

By value, the callee gets its own copy, and nothing is shared once the call returns. That suits small, plain data like a `Point`. Pass a pointer instead when the struct is large, when the callee should modify the caller's copy, or when C holds on to the struct after the call returns.

Wrapping a single value

`#[repr(transparent)]` is for structs with exactly one field. It guarantees that the struct has the same layout *and* the same function-call ABI as that field. That makes it useful for type-safe wrappers around values C knows as plain integers or pointers:

```rust
use std::ffi::c_int;

/// A file descriptor. C sees a plain `int`.
#[repr(transparent)]
pub struct Fd(c_int);
```

A one-field `repr(C)` struct has the same size and alignment as its field too, but some ABIs pass a struct differently from the bare value inside it. `repr(transparent)` guarantees they're passed the same way.

Head to the exercise

The exercise gives you a C header, and you write the matching Rust types from scratch. `Point` is a plain struct of two `doubles`. `Shape` holds an array, a nested `Point`, and a pointer that may be null. `Fd` wraps an `int`. The tests compare the size, alignment, and field offsets of each type against what the C compiler reports for the same header.

Exercise

The exercise for this section is located in `02_basics/03_structs` \newpage{=latex}

2.4 Enums and unions in FFI

A Rust enum is a sum type, with variants that can carry data. A C enum is just a set of named integers. Only field-less Rust enums map onto C enums. For variants with data, C pairs an integer tag with a union. This section covers how to handle each at the FFI boundary.

Field-less enums

In memory, a field-less Rust enum is just its discriminant: the integer that says which variant it is. `#[repr(C)]` gives the discriminant the size a C compiler would pick for the same enum.

```rust
#[repr(C)]
pub enum Color {
    Red = 0,
    Green = 1,
    Blue = 2,
}
```

In C, any integer fits in an enum type. You can assign 42 to an `enum Color`, and a newer version of a library can add variants your code has never heard of. In Rust, a value of an enum type must be one of its declared variants. An enum holding any other discriminant is undefined behavior. So when C sends you an enum value, declare the binding with the integer type, and convert it using `TryFrom`:

```
use std::ffi::c_int;
#[repr(C)]
pub enum Color { Red = 0, Green = 1, Blue = 2 }
unsafe extern "C" {
    fn current_color() -> c_int;
}

impl TryFrom<c_int> for Color {
    type Error = c_int;

    fn try_from(value: c_int) -> Result<Self, c_int> {
        match value {
            0 => Ok(Color::Red),
            1 => Ok(Color::Green),
            2 => Ok(Color::Blue),
            other => Err(other),
        }
    }
}
```

Unions

A Rust `union` works like a C union: all fields start at offset 0, and the union is as large as its largest field. Mark any union that crosses the boundary `#[repr(C)]`, since the default layout isn't guaranteed to match C's:

```
#[repr(C)]
pub union IntOrFloat {
    pub i: u32,
    pub f: f32,
}
```

A union doesn't know which of its fields holds a value. Writing a field is safe, but reading one is `unsafe`, because you're asserting that the bytes are valid for that field's type. Reading a `bool`, an enum, or a reference from bytes that hold something else can produce an invalid value, which is undefined behavior.

Union fields are also limited to types without a destructor, which in practice means `Copy` types, because the union can't tell which field's destructor to run.

Tagged unions from C

C often pairs an integer tag with a union, and relies on convention to keep the two in sync:

```
struct Value {
    int kind; /* VALUE_INT or VALUE_FLOAT */
    union {
        int64_t i;
        double f;
    } data;
};
```

Mirror that layout as it is, with a `repr(C)` struct and a `repr(C)` union, and keep the tag a `c_int`. Then convert into a Rust enum in one place, reading only the union field the tag points to:

```
use std::ffi::c_int;
pub const VALUE_INT: c_int = 0;
pub const VALUE_FLOAT: c_int = 1;
#[repr(C)] pub union CValueData { pub i: i64, pub f: f64 }
#[repr(C)] pub struct CValue { pub kind: c_int, pub data: CValueData }
pub enum Value {
    Int(i64),
    Float(f64),
}

fn read_value(value: &CValue) -> Option<Value> {
    // SAFETY: each arm reads only the field that `kind` marks as active.
    match value.kind {
        VALUE_INT => Some(Value::Int(unsafe { value.data.i })),
        VALUE_FLOAT => Some(Value::Float(unsafe { value.data.f })),
        _ => None,
    }
}
```

An unknown `kind` becomes `None` instead of undefined behavior, and the rest of your code works with a `Value` whose tag and data can't disagree.

To send a Rust enum to C in this shape, convert the other way: set the tag, and write the variant's data into the matching union field. Writing a union field is safe, so that direction needs no `unsafe` at all.

Head to the exercise

The exercise mirrors a C tagged union for you, next to the Rust enum the rest of the code works with. Convert that enum into the C struct, setting the tag and the matching union field for each variant.

Exercise

The exercise for this section is located in `02_basics/04_enums_and_unions` \newpage{=latex}

2.5 Slices in FFI

C has no slice type. A function that works on an array takes a pointer to the first element and, if you're lucky, the number of elements as a separate argument:

```
int sum(const int *items, size_t len);
```

Rust's slices are the same pair of pointer and length, packed into one value with guarantees attached. A `&[i32]` always points to `len` valid, initialized `i32`s, and the borrow checker makes

sure they stay that way while you use them. Turning C's pair into a slice is one of the most common operations in FFI code, and one of the easiest to get subtly wrong.

Slices from C

`std::slice::from_raw_parts` turns a pointer and a length into a slice, and `from_raw_parts_mut` does the same for a mutable one. The contract is longer than most people expect:

- The pointer must be **non-null and aligned**, even when the length is 0.
- It must point to `len` consecutive, **initialized** values of type `T`, all within a **single allocation**.
- The memory must **not be mutated** while the slice is in use. For `from_raw_parts_mut`, it must not be accessed through any other pointer at all.
- The total size, `len * size_of::<T>()`, must be at most `isize::MAX` bytes.
- The **lifetime** of the slice is whatever the caller picks, so you have to tie it to something that keeps the memory alive.

The first rule is the one that catches people. In C, passing `NULL` with a length of 0 is a normal way to say "no items". For `from_raw_parts`, it's undefined behavior, because a slice's pointer must never be null, not even an empty one. So handle the empty case before you build a slice:

```
use std::slice;

/// # Safety
///
/// If `len` is non-zero, `ptr` must be non-null and aligned, and point to `len`
/// initialized `T`s that stay alive and unmodified for `'a`.
pub unsafe fn slice_from_c<'a, T>(ptr: *const T, len: usize) -> &'a [T] {
    if len == 0 {
        return &[];
    }
    // SAFETY: `len` is non-zero, so the caller guarantees the whole contract.
    unsafe { slice::from_raw_parts(ptr, len) }
}
```

Everything else stays the caller's responsibility. If C says 10 and allocated 8, you read past the end, and the `# Safety` section is the place that says whose job it was to prevent it.

Debug builds check some of these rules at runtime and abort with `unsafe precondition(s) violated` when C passes `NULL`. Release builds skip those checks, and the same call can appear to work. Don't depend on this debug check.

Slices to C

Handing a slice to C is simpler. `as_ptr` and `len` give you the two halves, and `as_mut_ptr` gives you a `*mut T` for C functions that write into the buffer:

```
void scale_all(int32_t *items, size_t len, int32_t factor);
```

```
// Stands in for the C implementation, so that this example links.
#[unsafe(export_name = "scale_all")]
unsafe extern "C" fn scale_all_impl(items: *mut i32, len: usize, factor: i32) {
    let items = unsafe { std::slice::from_raw_parts_mut(items, len) };
    items.iter_mut().for_each(|item| *item *= factor);
}
unsafe extern "C" {
    fn scale_all(items: *mut i32, len: usize, factor: i32);
}

let mut items = [1_i32, 2, 3];
// SAFETY: the pointer and length describe `items`, which isn't accessed
// elsewhere during the call.
unsafe { scale_all(items.as_mut_ptr(), items.len(), 2) };
```

One thing to watch out for: The pointer of an empty slice is dangling: non-null, but not pointing to any memory. C code that checks `ptr != NULL` instead of `len != 0` will cause problems.

This only lends the memory to C for the duration of the call. That's also why there's no `into_raw_parts` to mirror `from_raw_parts`: a slice borrows its memory, so it has no ownership to give away. Handing ownership over is a different matter, which Allocation, allocators, and mixing them covers.

Head to the exercise

The exercise asks you to implement two functions that take a pointer and a length the way C would pass them: one that reads the items, and one that modifies them in place. The tests call them with normal input, and with both ways of saying "no items": a null pointer with a length of 0, and a valid pointer with a length of 0.

Exercise

The exercise for this section is located in `02_basics/05_slices` \newpage{=latex}

2.6 Opaque types

Some types can't cross the FFI boundary by value. A `Vec` is one of those, and so is a C `FILE` in the other direction. Neither can be mirrored field by field, because neither layout is guaranteed: Rust reserves the right to change `Vec`'s, and the C standard leaves `FILE`'s up to each C library.

Instead, pass a pointer across the FFI and keep the type behind it private. C calls this an opaque type, and it's the shape most C libraries already use: `FILE *`, `sqlite3 *`, `pthread_mutex_t *`. One side owns the object, its layout, and its memory, and the other side only ever holds the address. So an opaque type is really a question of ownership: who creates the object, and who frees it.

A Rust type behind a C handle

The C side gets a type name with no definition, and functions that take a pointer to it:

```c
typedef struct Counter Counter;

Counter *counter_new(uint32_t start);
void counter_add(Counter *counter, uint32_t amount);
uint32_t counter_value(const Counter *counter);
void counter_free(Counter *counter);
```

`typedef struct Counter Counter;` does two things. `struct Counter` declares a struct without defining it, and the `typedef` lets C code write `Counter` instead of `struct Counter`. The header never defines the struct, so C can pass `Counter *` around, but it can't allocate one, copy one, or read a field.

On the Rust side, `Counter` is an ordinary Rust struct. The constructor puts it on the heap and hands ownership to C, and the destructor takes it back:

```rust
pub struct Counter {
    value: u32,
}

#[unsafe(no_mangle)]
pub extern "C" fn counter_new(start: u32) -> *mut Counter {
    Box::into_raw(Box::new(Counter { value: start }))
}

/// # Safety
///
/// `counter` must be null, or come from `counter_new` and not be used again.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn counter_free(counter: *mut Counter) {
    if !counter.is_null() {
        // SAFETY: the pointer came from `Box::into_raw` in `counter_new`.
        drop(unsafe { Box::from_raw(counter) });
    }
}
```

`Box::into_raw` turns the box into a raw pointer without dropping the counter, and `Box::from_raw` turns the pointer back into a box, which drops the counter when the box goes out of scope.

The operations in between borrow the counter for the duration of one call. `&mut *counter` turns the raw pointer into a `&mut Counter`, and `&*counter` into a `&Counter`:

```rust
pub struct Counter { value: u32 }
/// # Safety
///
/// `counter` must come from `counter_new` and must not have been freed. No
/// other call may use the same counter for the duration of this one.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn counter_add(counter: *mut Counter, amount: u32) {
```

```
    // SAFETY: the caller guarantees a live counter that nothing else uses.
    let counter = unsafe { &mut *counter };
    counter.value += amount;
}

/// # Safety
///
/// `counter` must come from `counter_new` and must not have been freed. No
/// other call may modify the same counter for the duration of this one.
#[unsafe(no_mangle)]
pub unsafe extern "C" fn counter_value(counter: *const Counter) -> u32 {
    // SAFETY: the caller guarantees a live counter that nothing modifies.
    let counter = unsafe { &*counter };
    counter.value
}
```

These references are only sound because of the promises in the `# Safety` sections: the pointer is live, and nothing else uses the counter while the reference exists. A `&mut Counter` must be the only way to reach the counter, so two overlapping calls on the same counter would be undefined behavior.

Two rules keep this sound:

- **Pair every constructor with a destructor.** Every `counter_new` needs exactly one `counter_free`. After that, the pointer C holds is dangling.
- **Don't keep references past the call.** C may free the counter at any time, so a reference stored somewhere would outlive it.

A C type behind a Rust handle

In the other direction, a C library hands you a pointer to something you know nothing about:

```
typedef struct Widget Widget;

Widget *widget_open(int id);
int widget_id(const Widget *widget);
void widget_close(Widget *widget);
```

Don't guess its fields. Rust has a pattern for a type that stands for "something whose layout is none of my business":

```
use std::ffi::c_int;
use std::marker::{PhantomData, PhantomPinned};

#[repr(C)]
pub struct Widget {
    _data: (),
    _marker: PhantomData<(*mut u8, PhantomPinned)>,
}
```

```
unsafe extern "C" {
    fn widget_open(id: c_int) -> *mut Widget;
    fn widget_id(widget: *const Widget) -> c_int;
    fn widget_close(widget: *mut Widget);
}
```

Both fields are private, so the only way to get a `Widget` is a pointer from C. `_data` keeps the `improper_ctypes` lint quiet, which would otherwise flag a struct made only of `PhantomData`. The `*mut u8` in the marker makes the type neither `Send` nor `Sync`, so Rust won't assume it can be used from another thread. `PhantomPinned` makes it `!Unpin`, because the C library may rely on the object staying at the same address.

To make sure every widget is closed exactly once, wrap the pointer in a type that owns it. `NonNull` turns C's null into `None`, and `Drop` closes the widget when the wrapper goes out of scope:

```
use std::ffi::c_int;
use std::marker::{PhantomData, PhantomPinned};
use std::ptr::NonNull;
#[repr(C)]
pub struct Widget { _data: (), _marker: PhantomData<(*mut u8, PhantomPinned)> }
unsafe extern "C" {
    fn widget_open(id: c_int) -> *mut Widget;
    fn widget_id(widget: *const Widget) -> c_int;
    fn widget_close(widget: *mut Widget);
}
// Stand in for the C library, so that this example links.
#[unsafe(export_name = "widget_open")]
extern "C" fn widget_open_impl(id: c_int) -> *mut Widget {
    Box::into_raw(Box::new(id)).cast()
}
#[unsafe(export_name = "widget_id")]
extern "C" fn widget_id_impl(widget: *const Widget) -> c_int {
    unsafe { *widget.cast::<c_int>() }
}
#[unsafe(export_name = "widget_close")]
extern "C" fn widget_close_impl(widget: *mut Widget) {
    drop(unsafe { Box::from_raw(widget.cast::<c_int>()) });
}
pub struct OwnedWidget(NonNull<Widget>);

impl OwnedWidget {
    pub fn open(id: c_int) -> Option<Self> {
        // SAFETY: `widget_open` returns either a valid widget or null.
        let widget = unsafe { widget_open(id) };
        NonNull::new(widget).map(OwnedWidget)
    }

    pub fn id(&self) -> c_int {
        // SAFETY: the widget is still open, because `OwnedWidget` only closes
        // it in `drop`.
        unsafe { widget_id(self.0.as_ptr()) }
    }
}
```

```rust
impl Drop for OwnedWidget {
    fn drop(&mut self) {
        // SAFETY: the widget came from `widget_open`, and `OwnedWidget` is its
        // only owner, so it is closed exactly once.
        unsafe { widget_close(self.0.as_ptr()) };
    }
}
```

Head to the exercise

The exercise has both directions, each with a twist the examples above don't have. First, expose a Rust `Stats` type to C as a handle. Unlike the counter, it owns a `Vec` of its own, which has to be freed along with it. Then wrap a C `Buffer` in a Rust type that frees it exactly once.

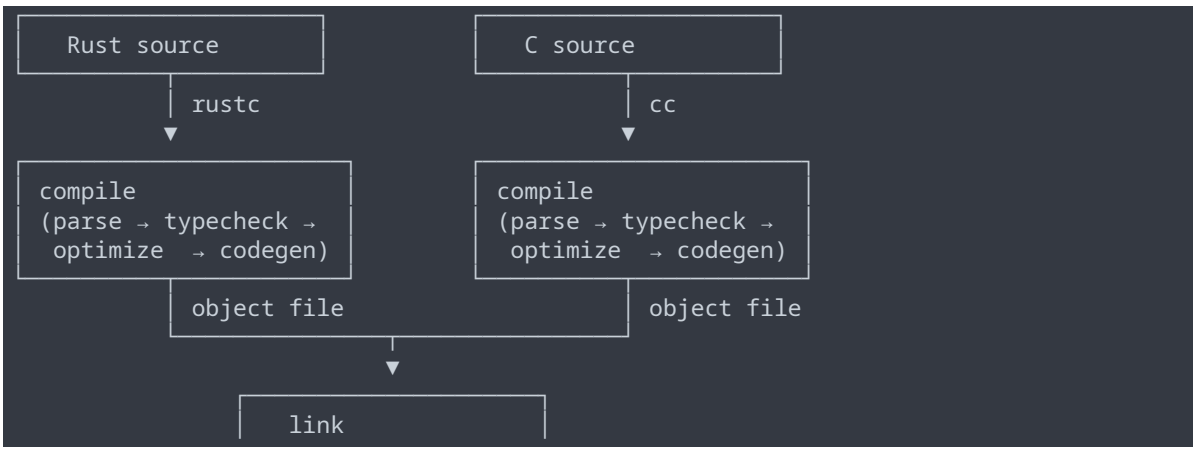
Exercise

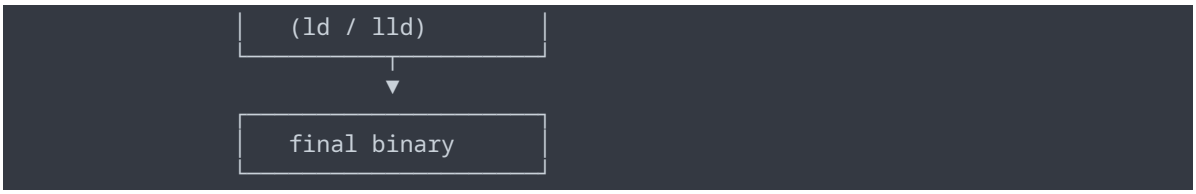
The exercise for this section is located in `02_basics/06_opaque_types` \newpage{=latex}

2.7 When hand-written bindings drift

In What is FFI? you wrote an `extern "C"` block binding to C code. It worked great, but can you imagine doing that for *hundreds*, maybe *thousands*, of functions and types? There's a subtler problem, too. Hand-written bindings will *inevitably* drift out of sync with the C header, especially at scale.

Neither the C nor the Rust compiler can detect this, because each operates in its own small universe, called a *compilation unit*. A compilation unit is the single chunk of work that flows through the various stages of the compiler. In C/C++ this would typically be a `.c`/`.cpp` file and in Rust that is typically a single crate. The compiler parses, typechecks, and optimizes the compilation unit, then produces an *intermediate* object file. Once all the compilation units that make up your project are built, the intermediate object files are gathered and passed to the *linker*, which produces the final executable or library.





This model is great for compilation performance, because we can process many of these compilation units *in parallel*. There is a catch: they *must not* share information, since that would destroy the parallelism. Each compilation *must* be its own self-contained universe.

And even if we had a mechanism to share information between compilation units, we would need to make it language-agnostic so that a C compilation unit and a Rust compilation unit can interoperate. How would that even work with wildly different type systems?[3]

So compilers have resorted to *manual escape hatches* like the `extern "C"` block you wrote. You as the programmer promise to the compiler that a function with a given name and a given signature will exist at link time, and the compiler takes your word for it.

Possible consequences

The consequences of drift between the two versions can be dire: if C expects `int` but Rust expects `float`, the integer bit patterns are reinterpreted as floats. The result is almost always nonsensical.

It can be worse: passing too many or too few arguments can result in either overwriting important information on the stack or reading garbage from it.

Here is an especially insidious case. The function, as we have established, is the following:

```
int bm_add(int a, int b);
```

but now in the Rust code, we expect it to return 64-bit integers instead of 32-bit integers:

```
use std::ffi::c_longlong;
unsafe extern "C" {
    fn bm_add(a: c_longlong, b: c_longlong) -> c_longlong;
}
```

How will this example fail? If you try it yourself, you will notice that it doesn't. Function arguments are passed in registers, and on a 64-bit machine those registers are 64 bits wide, so a 32-bit `int` occupies one of them with room to spare. What the upper half of the register holds is not something the declaration can rely on: some ABIs require a narrow integer to be sign-extended across the full register, others leave the high bits unspecified entirely. Nothing widens the argument on your behalf. Reading it back as a 64-bit integer just happens to give the right answer here.

[3] Yes, the LLVM bitcode embedded by toolchains for LTO (Rust's `-Clto=thin -Cembed-bitcode=yes` and Clang's `-flto=thin`) *does* carry the information to catch problems like this at link-time and it *is* cross-language, but linkers do not generally validate it. (The `wasm-ld` linker does, but only for Wasm.) The reasons are many, the most important being that validating it would break existing compiler optimizations. You *could* write custom LLVM-bitcode parsing tooling to check this if you wanted.

You can see this in Compiler Explorer here: The correct version passes a and b in the a0 and a1 registers (the li a0, 1 and li a1, 2 lines do the loading) and passes the return value in the a0 register. Those registers are 64 bits wide, so you can see the incorrect c_longlong-expecting version **just works**: the values are small and positive, and nothing has put anything else in the upper halves of a0 and a1.

Now, if your 64-bit numbers stay below the 32-bit max value, everything just happens to work. But it is *very* fragile. As you can see here, when we compile to a 32-bit target instead, everything breaks and we end up adding 0 to a instead.

This example is particularly scary, because for 99% of inputs and deployment configurations the mistake is virtually consequence-free. The addition will continue to work as expected. But as soon as the input is unusual, the deployment target is different, *or you add code that will make the compiler change the generated code even a bit*, this will be a bug that takes you weeks to troubleshoot in the worst case.

Head to the exercise

The exercise turns this around: the Rust binding stays fixed and correct-looking while the C implementation drifts underneath it. It is the same mismatch, and therefore the same bug, from either side. Drifting the C side just keeps the Rust crate compiling, so you can run it and watch what happens.

Play with the different "drifted" C implementations of bm_add to see what happens on your machine. Feel free to also play around a bit with the Compiler Explorer playgrounds to see how different miscompilations manifest in the generated assembly.

Exercise

The exercise for this section is located in 02_basics/07_drift \newpage{=latex}

2.8 Generating Rust bindings with bindgen

In the previous exercise you saw how easy it is for manual bindings to go out of sync, and the scary silent corruption that causes. This section introduces a tool that avoids all of this: **bindgen**. It's a library you call from your Rust build script to generate Rust bindings to C code from C header files. It runs libclang on a C header and emits matching Rust declarations: extern blocks, #[repr(C)] structs, integer constants. The header remains the single source of truth, with the Rust bindings being generated on every build.

You call bindgen from your build script like so:

```
let bindings = bindgen::Builder::default()
    .header("c_src/bm_legacy.h")
    // tell Cargo to re-run when headers change
    .parse_callbacks(Box::new(bindgen::CargoCallbacks::new()))
    .generate()
    .expect("bindgen failed");
```

```
let out_path = PathBuf::from(env::var("OUT_DIR").unwrap());
bindings.write_to_file(out_path.join("bindings.rs")).unwrap();
```

This writes the generated file into the crate's `OUT_DIR` (usually `target/<debug or release>/build/<crate name and hash>/out`). Inspecting the generated file, you will see something like this:

```
/* automatically generated by rust-bindgen 0.72.1 */

unsafe extern "C" {
    pub fn bm_add(a: ::std::os::raw::c_int, b: ::std::os::raw::c_int) -> ::std::os::raw::c_int;
}
```

You can then pull this generated file into your crate with the `include!` macro like so:

```
mod sys {
    include!(concat!(env!("OUT_DIR"), "/bindings.rs"));
}
```

This is great: we've gained end-to-end type checking. A change in the C header will not silently corrupt our Rust code. But remember, these bindings are as `unsafe` as the C code itself. It's your responsibility to use them correctly.

You would typically wrap them in a safe, idiomatic API. This is the common `-sys` crate pattern: a `<crate>-sys` crate holds the raw bindings, while the `<crate>` crate on top exposes the safe abstractions.

Head to the exercise

Replace the hand-written `extern` block from the previous section with `bindgen` in `build.rs`. The tests stay identical.

Tip

`cargo expand -p bm_bindgen` shows what `bindgen` produced. Requires `cargo-expand`.

Exercise

The exercise for this section is located in `02_basics/08_bindgen` \newpage{=latex}

2.9 Exposing Rust to C with cheadergen

With `bindgen` we generated the Rust declarations for calling C. Now we go the other way: our goal is to *migrate* C code to Rust, so C will need to call into our new Rust code.

`extern "C" fn` and `#[unsafe(no_mangle)]` take care of the calling convention and the symbol name:

```rust
use std::ffi::c_char;

#[unsafe(no_mangle)]
pub extern "C" fn bm_strlower(s: *mut c_char) {
    // ...
}
```

What's still missing is a header file that tells the C compiler about the API. You could write it by hand, but you'd be maintaining two sources of truth. **`cheadergen`** (the inverse of `bindgen`) generates C headers from Rust code.

`cheadergen`, created and maintained by our colleague Luca Palmieri, reads your Rust code and generates a C header for the relevant FFI-visible symbols.

Install the CLI from the `cheadergen` releases page.

Then you can run the following command in your crate root (for example, the current exercise crate):

```
cheadergen generate
```

and a header will appear:

```
/* Generated by cheadergen — do not edit */

void bm_strlower(char *s);
```

`cheadergen` comes with two ways to configure its output. The first is a project-wide `cheadergen.toml` file. Here you can tweak the style of the generated headers, include custom preamble text, and more. See the full documentation here.

The second configuration option is the `#[cheadergen::config(...)]` attribute. You can apply this attribute to any Rust type. It lets you configure per-type options such as renaming fields, skipping exporting, forcing exporting, and more. See the full documentation here.

A note on `cbindgen`

If you've been researching Rust FFI tooling, you've undoubtedly come across `cbindgen`, a similar tool maintained by Mozilla, and you may be wondering: "Why did you just introduce me to this custom tool instead?"

Well, we wouldn't have built a new tool if the existing one fit. `cbindgen` dates back to 2017, when the Rust ecosystem looked quite different. To read your Rust and emit headers, it had to parse the source itself with a custom `syn`-based parser. At the time, that was the only way, since the compiler didn't expose enough information.

But the technique has real limits. The custom parser doesn't always agree with rustc, it often needs manual steering and fixups, and when it hits something it can't handle it tends to silently skip generating a header rather than tell you why. None of this is a knock on `cbindgen`: it's what the constraints of the era allowed. `cheadergen` exists because those constraints have since eased, letting us build something with better diagnostics and fewer surprises.

Head to the exercise

You'll port the `bm_strlower` function to Rust and call `cheadergen generate --lang c --output-dir c_test -p bm_cheadergen` to generate the header automatically.

Exercise

The exercise for this section is located in `02_basics/09_cheadergen` \newpage{=latex}

2.10 The FFI boundary as a firewall: validate and narrow

By now you've written `extern "C"` functions in both directions, seen `bindgen` generate Rust code and `cheadergen` generate C headers.

On paper that is everything you need, and it's tempting to go ahead and rewrite your project now. Take your C API and translate it one-for-one into Rust.

The problem is that often you can't, and just as often you shouldn't. Real-world C is complicated, idiosyncratic, and (let's admit it!) full of skeletons. You're likely thinking about a Rust rewrite not just to address security problems or performance issues; rewriting in Rust is a chance to clean up your codebase: re-establish module boundaries, drop legacy assumptions, write the module you wished you had.

At the same time big-bang rewrites (where you replace the entire codebase at once) famously never work. Which leaves a problem: you have a clean Rust codebase, a messy legacy codebase, and a need to keep both working together for *some time*[4].

During this transitional period, the FFI layer is where those worlds meet. It has to bridge unsafe-everything-goes C and the borrow checker, and bridge your new design and the old one.

Many people think FFI is about *type translation*. That's true as far as it goes, but it's like saying programming is about typing words into a computer: it misses the point. The *main* job of your FFI interface is **establishing confidence**. In a legacy codebase you rarely know with 100% certainty that reality matches your assumptions: some caller might actually pass a null pointer, or the `*const c_char` that used to be UTF-8 might now hold binary data.

The FFI boundary is your chance to turn legacy uncertainty into known-good state before it crosses into your new system. Uncertainty leaks. Your FFI must be a vigilant firewall against it.

To this end, we've settled on a handful of rules that we now live by.

Always validate your inputs

Validate every assumption at the FFI boundary. Null checks, length checks, UTF-8 checks. Panic or return an error if anything is even slightly off. In web servers, the HTTP handler function is

[4] The exact amount of time of course varies with the scale of your codebase, but from hard-won experience: measure this in years. C codebase migrations have a funny habit of taking much, much longer than you think. As a rule of thumb: take your worst-case estimate, double it, and add a year. Really.

your primary interface to the "chaos world" of the internet. Think of FFI functions the same way: they are your primary interface to the legacy-C "chaos world".

You should also lean on Rust's type system so "forgot to check" isn't even an option. For example: at Mainmatter, we encourage contributors to use `Option<NonNull<T>>` instead of `*mut T` as much as possible, for the simple reason that a `*mut T` can be dereferenced directly (`*ptr`), which triggers UB if the pointer is null. You would have to add a manual `if ptr.is_null() {}` check before every pointer dereference. With `Option<NonNull<T>>`, on the other hand, the type system *forces* you to handle the None case explicitly. Even the laziest `.unwrap()` will result in a loud panic instead of potentially silent UB.

```
// `*mut T` silently accepts null. You'll remember to check. Until you don't.
pub extern "C" fn bm_do_thing(input: *mut Thing) -> BmResult { /* … */
}

// `Option<NonNull<T>>` encodes null as `None` — the compiler forces the branch.
pub extern "C" fn bm_do_thing(input: Option<NonNull<Thing>>) -> BmResult {
    let Some(input) = input else {
        return BmResult::ErrInvalid;
    };
    // `input: NonNull<Thing>`, statically non-null.
    /* … */
}
```

Don't overload primitives

Legacy C code has a habit of packing several meanings into one return type. POSIX `read()` is a good example: `-1` means error (check `errno`), `0` means end-of-file, anything positive is the byte count. It is *very easy* to mess this up. Rust gives you better tools: `bool` for yes/no, an enum for branching outcomes, an out-parameter for the count. These types usually translate well into C headers too, so use them.

```
#[repr(C)]
pub enum BmReadStatus { Ok, Eof, IoError }

pub extern "C" fn bm_read(/* … */, out_bytes: Option<NonNull<usize>>) -> BmReadStatus;
```

Head to the exercise

We'll continue the porting work by looking at `bm_normalize_url`, in `exercises/_bm/src/normalize.c`. It will normalize a given URL by lowercasing it and writing the normalized string into the provided buffer. The exercise already contains our *"first draft"* Rust translation: a naive transliteration from C to Rust that has a number of issues.

Let's apply what we learned above and fix the implementation.

Hint: check the exercise's tests for the expected behavior of `bm_normalize_url`.

Exercise

The exercise for this section is located in `02_basics/10_validate_and_narrow` \newpage{=latex}

2.11 High-quality FFI APIs

We previously introduced two "mechanical" rules: always validate your inputs, and don't overload primitives. The two that follow are more conceptual. Don't let that fool you: they are just as important as the first two, if not more so.

Documentation, documentation, documentation

Some invariants you can check at runtime. Many you can't: who owns this pointer, whether the string is copied or borrowed, whether `free` has already run, what each error variant means. Write them all down.[5] Rust's `# Safety` convention, being just a comment, works with `extern "C"` functions too, and `cheadergen` emits those sections as C doc comments in the header files:

```
/// Normalize a URL into the caller's buffer.
///
/// # Ownership
/// `url` and `out` are borrowed for the call; the caller continues to own both.
///
/// # Safety
/// 1. `url`, if non-null, must be a [valid] pointer to a NUL-terminated UTF-8 byte sequence.
/// 2. `out`, must be a [valid], non-null pointer to a writable buffer of at least `out_len` bytes
///
/// # Errors
/// - `BmResult::ErrInvalidUrl` if `url` is null or not valid UTF-8.
/// - `BmResult::ErrBufferTooSmall` if the result wouldn't fit in `out_len` bytes.
///
/// [valid]: https://doc.rust-lang.org/std/ptr/index.html#safety
#[unsafe(no_mangle)]
pub unsafe extern "C" fn bm_normalize_url(
    url: Option<NonNull<c_char>>,
    out: Option<NonNull<c_char>>,
    out_len: usize,
) -> BmResult {
    let Some(out) = out else {
        //...
    };

    // SAFETY: caller ensured `out` points to at least `out_len` bytes (2.)
    unsafe { slice::from_raw_parts_mut(out.as_ptr(), out_len) };
}
```

[5] You may say, jadedly, that no one ever reads comments, and you may be right. But with the rise of LLMs, *something* does read them. We've found that LLMs struggle with the nuanced, unspoken invariants of FFI code, which is not surprising. Turning as many of these unspoken invariants into spoken ones helps you get better mileage out of these tools.

Note how the safety invariants are numbered. A plain `# Safety` section lists what the caller must uphold; numbering the entries lets every inline `// SAFETY:` comment point at the one it relies on. We require each of those comments to either:

- delegate its local invariant to the surrounding function's safety comment, in which case it must reference a numbered invariant; or
- explain exhaustively why the code itself upholds the local invariant.

This way we make sure that all invariants are either upheld by the function itself or correctly documented as a responsibility of the caller.

At the moment, the numbered-invariant convention is checked by discipline and PR review. Clippy can require `# Safety` documentation and comments on `unsafe` blocks, but it cannot verify that a local justification actually upholds an invariant. There are proposals for tools that could make this kind of traceability machine-checkable in the future.[6]

Mind the FFI tax

Every exposed function is API surface you'll maintain forever, an `unsafe` contract to keep correct, and a per-call cost the compiler can't optimize away. Cross-language LTO can inline across the boundary, at the cost of a real setup burden. The cheapest FFI function is the one you didn't expose. Prefer coarse operations, such as `bm_thing_update_with(...)`, over one setter per field, and treat the boundary as a small set of verbs, not a mirror of your internal struct.

Head to the exercise

You'll update your solution to exercise 2.10 with the two rules from this section.

Exercise

The exercise for this section is located in `02_basics/11_high_quality_ffi_apis` `\newpage{=latex}`

[6] There are a couple of related proposals, all in the "pre-RFC" stage. The most interesting is the safety-tags pre-RFC.

Chapter 3

Chapter 3: Intermediate

From now on we will be porting a "real" C application: the `bm` bookmark manager CLI. We will learn how to approach real-world codebases, structure our approach, and debug our code when it breaks.

In this chapter we'll pick our first module to rewrite, learn how C and Rust allocators (don't) mix, debug across the language boundary with `lldb`/`gdb`, harden our `unsafe` code with Clippy lints, replace C-isms with Rust-isms, catch undefined behavior with Miri, and benchmark the port against the C baseline.

Exercises

The exercises for this chapter are located in `exercises/03_intermediate`

Exercise

The exercise for this section is located in `03_intermediate/00_README` \newpage{=latex}

3.1 How to rewrite a module

In the previous chapter you learned the FFI building blocks: `extern` blocks, generated bindings, FFI-safe types, and how to design a boundary that acts as a firewall. Now we'll put them to work on a real codebase. Meet `bm`, a small bookmark manager CLI written in C.

Here is an overview of the C source code:

```
exercises/_bm/src/
├── bookmark.c    the Bookmark type
├── bookmark.h
├── cli.c         entry point, argument parsing
```

```
├── index.c       in-memory bookmark index
├── index.h
├── internal.h    definitions shared by index.c and storage.c
├── normalize.c   URL and tag normalization
├── normalize.h
├── result.h      shared result codes and error-string helper
├── storage.c     on-disk persistence of .bm files
├── storage.h
├── tag.c         tag parsing and matching
├── tag.h
├── util.c        string helpers
└── util.h
```

Picking the first module

There is no universally best place to start. A leaf module is often a good choice because it has few dependencies. The program's entry point can also work if it mostly calls other modules through simple interfaces. Sometimes the best boundary does not match an existing source file at all, and it is worth extracting a small module before porting it.

The right choice depends on the codebase, but a few guidelines help:

- Prefer a small, clearly defined interface.
- Minimize the number of C functions and data structures the Rust code must use.
- Avoid shared global state and complicated ownership rules in the first port.
- Choose code with useful tests, so you can compare behavior before and after.
- Keep the change small enough to review and, if necessary, revert.

```
cli       → bookmark, index, normalize, result, tag, util
index     → bookmark, internal, normalize, storage, tag, util
storage   → bookmark, index, internal, normalize, util
tag       → normalize, util
bookmark  → util
normalize → (no project dependencies)
util      → (no project dependencies)
```

Two things are worth noting. `normalize` and `util` are the only real leaves: every other module pulls in at least one of its siblings. And `index` and `storage` include each other, so this is not the clean tree the file names suggest. Tangles like that are normal in code that has been maintained for a while, and they are exactly what you want to know before choosing where to start.

The exercise uses an even smaller boundary than these production modules: a one-function `bm_version` module. Its behavior is intentionally trivial so that the work is limited to replacing a C object file with a Rust static library.

Preserve the existing contract

When possible, an incremental migration replaces the module's *object file* without changing the interface used by the remaining C code. The existing C header describes the ABI our Rust implementation must initially satisfy. The exercise's header contains this declaration:

```c
#include <stdint.h>

uint32_t bm_version(void);
```

The Rust replacement must export the same function under the same symbol name and use the C ABI:

```rust
#[unsafe(no_mangle)]
pub extern "C" fn bm_version() -> u32 {
    1
}
```

The rest of the C code doesn't know, and doesn't need to know, that the implementation behind the symbol changed. The linker resolves the same symbol names as before; it just finds them in our Rust static library instead of the old object file.

That is a migration constraint, not necessarily the interface we want in the long term. Some C headers expose globals, macros, shared data structures, or ownership assumptions that cannot be reproduced cleanly in Rust. In those cases, we can introduce a small compatibility layer and improve the interface separately.

After that, all you have to do is link against the replacement library written in Rust instead of the original one written in C. Cargo must first produce the right kind of artifact:

```toml
[lib]
crate-type = ["staticlib"]
```

```
before:  test_bm_version.o + bm_version.o (C)
after:   test_bm_version.o + libbm_version.a (Rust)
```

Structuring the Rust side

For a substantial module, keep the two layers from the firewall pattern in section 2.10: a thin `extern "C"` surface that converts C representations and a safe Rust core that implements the behavior. `bm_version` takes no arguments and returns a fixed-width integer, so in this case the exported function can remain the entire implementation.

Verifying behavior parity

A rewrite must not change observable behavior. The exercise checks this by running the same C caller twice:

1. first it links the caller with `bm_version.c` and runs the all-C baseline;
2. then it omits `bm_version.c`, links the unchanged caller with the Rust archive, and runs it again.

A Rust unit test independently checks the function's return value. The C link is what proves that the archive exposes the ABI and symbol promised by the header.

Tips

If the linker complains about an undefined symbol, inspect the symbols your Rust static library exports. The command depends on your toolchain and platform:

- Unix-like systems: `nm target/debug/lib<crate>.a`
- LLVM toolchains: `llvm-nm target/debug/lib<crate>.a`
- Windows with MSVC: `dumpbin /symbols target\debug\<crate>.lib`

A missing `#[unsafe(no_mangle)]` is a common cause.

While both implementations still exist, you can also run the same inputs through the C and Rust versions and compare their results. This is called differential testing. The exercise uses the simpler form of running the same assertions against both implementations.

Head to the exercise

The exercise in `exercises/03_intermediate/01_rewrite_a_module` replaces `bm_version.c`. Configure the crate to produce a static library, then make the provided Rust function satisfy `bm_version.h` without changing the header or the C caller. There is no algorithm to port: the exercise focuses on the build artifact, ABI, symbol name, and link step.

Exercise

The exercise for this section is located in `03_intermediate/01_rewrite_a_module` \newpage{=latex}

3.2 Allocation, allocators, and mixing them

In the previous section you saw how to replace a whole C module with Rust while keeping the interface identical. But `bm` passes more than integers and borrowed strings across the boundary: it passes *ownership*. `bookmark_new` allocates a `Bookmark` that someone else will free later. Who frees it, and with what? This is where mixed-language codebases hide some of their nastiest bugs and most difficult design challenges.

The golden rule

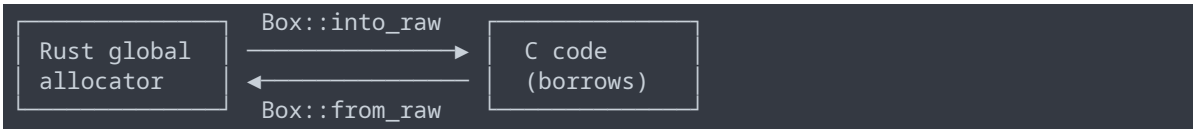
Every allocation must be released through its corresponding deallocation API.

That's the whole rule. C's `malloc`/`free`[1] and Rust's global allocator are two independent bookkeeping systems[2]. Handing a Rust-allocated pointer to C's `free` (or a `malloc`'d pointer to

[1] `malloc`/`free` are standard C allocation functions, but nothing prevents you from using other allocation functions or even rolling your own. For the sake of simplicity, we will always refer to `malloc`/`free` in this section, but read it as `malloc`/`free` and all other custom (de)allocation functions.

[2] Currently, Rust's default global allocator is unspecified, but on many platforms it *is* the system `malloc`. Libraries such as cdylibs and staticlibs, however, are guaranteed to use the `System` allocator by default. Beware though: C's `malloc`/`free` API and Rust's global-allocation API are not interchangeable, even when both happen to use the same underlying system allocator.

`Box::from_raw`) is undefined behavior, even if it happens to "work" on your machine because both forward to the same underlying `malloc` today. They might stop doing so tomorrow, or not do so at all on a different architecture.



Crossing the boundary in practice

There are several workable strategies:

1. **Use C allocation APIs from Rust.** Rust can call a C constructor such as `bookmark_new` and wrap the returned pointer in an owning Rust type. Its `Drop` implementation releases the allocation by calling the matching C function, `bookmark_free`. Keep allocation and deallocation within the same C API; export the functions through FFI to Rust.

   Opaque types shows the shape of such a wrapper.

2. **Use Rust allocators from C.** A Rust type that crosses the boundary will need a matching `_new`/`_free` pair exported through FFI. `_new` in Rust is paired with `_free` in Rust; C only ever holds the pointer and calls the Rust functions to allocate or free an object of that type.

   The basic pattern for this is to convert a `Box` to and from a raw pointer with `into_raw` and `from_raw`:[3]

   ```
   let foo = Box::new(42);
   let ptr = Box::into_raw(foo);
   // This will convert the raw pointer back into a Box and when dropped free the memory:
   let _foo = unsafe { Box::from_raw(ptr) };
   ```

   Opaque types shows the full `_new`/`_free` pair.

3. **Transfer ownership of a C allocation across the boundary.** Use this when one language calls a C allocation API, transfers ownership of the result to the other language, and the receiving side later calls the matching C deallocation API. For example, Rust can allocate a NUL-terminated string with libc::malloc and pass it to C code that retains it and later calls `free`.

   The reverse direction is valid too: C can transfer ownership of a value it allocated, for example with `malloc`, to Rust, and Rust can later call the matching deallocation function through FFI. In both cases, the allocation and deallocation calls originate on opposite sides of the FFI boundary. The layout and ownership contract must exactly match what the receiving API expects.

4. **Use a `malloc`/`free` adapter as Rust's global allocator.** On Linux, Rust's default global allocator typically already uses the system allocator, so this is usually unnecessary. It can still be useful when Rust must use a specific C allocation API. Implementing the adapter is

[3] There are some other useful structs, besides `Box`, that come with their own `from_raw` and `into_raw` functions, like `Vec` or `CString`. The same pattern as for `Box` applies here: allocate in Rust → convert into raw pointer and transfer ownership to C → use the pointer in C → free the pointer by transferring ownership back to Rust.

non-trivial: it's inherently `unsafe`, and you need to carefully uphold all the safety guarantees required by the GlobalAlloc trait. Do not pass Rust-owned values to C for it to free. Even with a `malloc`/`free` global allocator, C cannot run Rust destructors; freeing a value containing a `String`, `Vec`, or other owned resource leaks what that value owns.

Avoid if possible. An allocation adapter is trickier than it seems. The code is small, but getting its allocation contract right requires a lot of implicit knowledge. Only use one when you have a specific interoperability need.

5. **Use the Allocator API.** As of this writing, the API is still experimental and only available on nightly Rust. It functions similarly to `GlobalAlloc`: you have to implement your own allocator, but unlike the global one, it is a lot more flexible and can be used on a per-variable basis. The new API adds an Allocator trait and `Allocator` type parameters to many allocation-owning types, such as `Box` or `Vec`.

```
pub struct Box<T, A = Global>()
// Global is the global allocator implementing the Allocator trait
where
    A: Allocator,
    T: ?Sized;
```

Additionally, it adds a few new functions where you can pass an allocator, for example `new_in`:

```
#![feature(allocator_api)]

use std::alloc::System;
let foo = Box::new_in(1, System);

let custom_alloc = SomeCustomAlloc::new();
let bar = Box::new_in(2, custom_alloc);
```

`foo` and `bar` now use different allocators.

Avoid if possible. The same warning as for `GlobalAlloc` applies here: implementing an allocator is a non-trivial undertaking and is only worth it if you have a specific use case.

This API is primarily useful for arena or bump allocators, where many related values are allocated from the same memory region and released together. Another useful case is selectively integrating parts of your program with a different allocation system while still using standard owning types such as `Box` and `Vec`.

Spotting allocator mismatches

Tools like Valgrind give every allocation a birth certificate: run the test binary under `valgrind --leak-check=full` and it will tell you not just that a block leaked, but which call stack allocated it.

Valgrind has no Apple Silicon support, so on an M-series Mac reach for `leaks`, for AddressSanitizer via `-Zsanitizer=address`, or, for the Rust side of the boundary, for Miri, which we cover in a later section.

Head to the exercise

In `exercises/03_intermediate/02_allocators`, you'll write a global allocator that wraps the system allocator and counts every allocation and deallocation. Then you'll use it to track down a leak: a pair of functions hands strings to C, and one of them never gives the memory back.

Exercise

The exercise for this section is located in `03_intermediate/02_allocators` \newpage{=latex}

3.3 Using `lldb/gdb`

At some point in a migration, you'll hit a crash at the FFI boundary with little to go on. A C caller passes a string without the NUL terminator the API requires, or Rust hands C a pointer whose lifetime has ended. The process takes a `SIGSEGV` and dies before Rust can give you a tidy panic message or backtrace.

gdb and `lldb` can show the mixed C/Rust stack. Set a breakpoint in Rust, step into C, and inspect both sides of the boundary in one session.

Build with debug information

Build both sides of the boundary with debug information. Cargo's `dev` profile already sets `debug = true`; for a release build, add this to `Cargo.toml`:

```
[profile.release]
debug = true
```

If `build.rs` compiles the C code with `cc`, pass `-g` there too:

```
cc::Build::new()
    .file("c_src/shout.c")
    .include("c_src")
    .flag_if_supported("-g")
    .compile("shout");
```

Forget it and you get Rust frames with line numbers and C frames without: precisely the half of the stack you opened the debugger for.

Start the debugger

Use `rust-gdb` or `rust-lldb` rather than plain `gdb` and `lldb`. They're thin wrappers that load Rust's pretty-printers, which is what makes a `String`, `Vec`, or `Option` readable instead of a pile of pointers and capacities.

On macOS, that's usually `rust-lldb`:

```
rust-lldb target/debug/bm_lldb_gdb
```

On Linux, `rust-gdb`:

```
rust-gdb target/debug/bm_lldb_gdb
```

Program arguments go after `--` for `rust-lldb` and after `--args` for `rust-gdb`.

If you prefer a graphical frontend, `rust-gdbgui` starts the browser-based `gdbgui` with the same pretty-printers loaded. Install `gdbgui` separately.

```
rust-gdbgui target/debug/bm_lldb_gdb
```

Walk across the boundary

If you already know one debugger and need the equivalent command in the other, keep the official GDB to LLDB command map open. It covers both directions in one place.

Say you're investigating `bm_shout`, the C helper called by the Rust exercise. Set a breakpoint on it, run the program, and look at the stack when execution stops (output abbreviated):

```
(lldb) b bm_shout
Breakpoint 1: where = bm`bm_shout at shout.c:6
(lldb) run
Process 51234 stopped
* thread #1, stop reason = breakpoint 1.1
  * frame #0: bm`bm_shout at shout.c:6
    frame #1: bm`bm_lldb_gdb::shout at lib.rs:17
    frame #2: bm`bm_lldb_gdb::demo at lib.rs:30
    frame #3: bm`main at main.rs:2
```

C frames and Rust frames are interleaved, and the numbering tells you which way to walk: frame #0 is where execution stopped, and every higher number is one of its callers. `up` moves toward callers, `down` back toward the stop. So the code that handed over the bad value is always *up*, whichever language crashed. If Rust crashed, `up` gets you to the C caller and the arguments it passed. If C crashed, `up` gets you to the Rust frame that supplied them.

From there the usual commands work in both debuggers: `s` steps into a call, `n` steps over one, `finish` runs the current frame to completion, `bt` prints the stack, and `p` prints a value.

For a `char *out`, inspect the bytes directly: `x/s out` in gdb, and either `x/s out` or `memory read --format c-string out` in `lldb` (the format name is `c-string`, with the hyphen; `cstring` is rejected). For a raw buffer, use `x/16xb out` in gdb or `memory read --size 1 --count 16 --format x out` in `lldb`. This quickly tells you whether the pointer, length, and terminator agree with the FFI contract.

When the process dies outright, you don't need a breakpoint at all. Run it, let it take the signal, and print the stack:

```
(lldb) run
Process 51234 stopped
* thread #1, stop reason = EXC_BAD_ACCESS (code=1, address=0x0)
(lldb) bt
```

The frame you land in is where the bad pointer was dereferenced, which is not necessarily where the bug is. Walk up until the values stop making sense.

A note on Rust symbol names

Everything above assumes your debugger can read the names in that stack. Since Rust 1.97, `rustc` uses the v0 symbol-mangling scheme by default. Nightly has done so since November 2025, so the toolchain this book pins already emits v0 symbols.

Symbol mangling is how the compiler turns a function into a linker-safe name. For a C function, that's usually just the function name. Rust also has to distinguish generic instantiations: `Vec<u8>::push` and `Vec<String>::push` are different generated functions and therefore need different symbols.

The old scheme used a hash for much of that distinction. It worked, but a backtrace through generic code often left you with a readable function name and an opaque suffix. The v0 scheme preserves the concrete generic arguments in a form tools can decode, so when you're debugging or profiling a monomorphized call chain, the backtrace can show which instantiation actually ran rather than a hash.

This doesn't affect functions exported to C with `#[unsafe(no_mangle)]`, but it does require every tool that reads Rust symbols to understand v0. Recent `gdb`, `lldb`, and Rust tooling do. Older debuggers, profilers, or crash-symbolication pipelines may show raw names such as `_RNvNtNtCs...` instead. Check your whole debugging path before upgrading a pinned CI or production toolchain.

A quick test is enough: write a small generic function that panics, run it with `RUST_BACKTRACE=1`, and read the backtrace.

See the stabilization PR and the v0 symbol-format documentation for details.

Head to the exercise

In this exercise a small C helper and its Rust binding disagree about something that only shows up once you call across the boundary. Build it with `cargo build -p bm_lldb_gdb`, then debug `target/debug/bm_lldb_gdb`. No `println!` allowed.

Exercise

The exercise for this section is located in `03_intermediate/03_lldb_gdb` \newpage{=latex}

3.4 Clippy lints for unsafe Rust and static verification

In the previous section we learned how to debug an application, but debugging only gets you so far: it starts from a crash you already have, and works backward to the root cause.

Lints, on the other hand, help prevent certain issues in the first place. They catch a narrower set of problems and suspicious patterns before the code runs. Running them can be fully automated, which helps enforce coding standards across an entire team.

They are especially useful in `unsafe` code. They can require each unsafe operation to be explicit, flag signatures that don't state the caller's obligations, and surface undocumented assumptions. They can't prove that a pointer is valid or that a C caller is honest, but they can make unchecked assumptions visible for review.

Built-in lints

You already have several useful lints enabled by default.

`improper_ctypes` and `improper_ctypes_definitions` are rustc lints, warn-by-default, and you've encountered them already: they're what complained about the non-FFI-safe types back in Chapter 2. `improper_ctypes` covers the types you declare in an `extern` block; `improper_ctypes_definitions` covers the ones you define with `extern "C"`. Between them they catch most type-level mistakes long before a linker gets involved.

`clippy::not_unsafe_ptr_arg_deref` is a correctness lint, so Clippy already marks it as deny rather than warn. It fires when a public safe function dereferences a raw-pointer argument even though its signature places no requirements on the caller. Either make the function unsafe and document its pointer contract, or keep it safe and accept a type that can be dereferenced safely. Don't silence the lint.

`clippy::missing_safety_doc` flags a public `unsafe fn` whose docs have no `# Safety` section. Set `check-private-items = true` in `clippy.toml` to apply it to private items as well. This is worth doing in an FFI crate, where plenty of the interesting unsafe code isn't public.

More lints to turn on

You can configure a lint in code:

```
#![warn(clippy::undocumented_unsafe_blocks)]
```

Or you can use lint tables in `Cargo.toml`. We'll use `Cargo.toml` here to keep this crate-wide policy in one place:[4]

```
[lints.rust]
unsafe_op_in_unsafe_fn = "deny"

[lints.clippy]
undocumented_unsafe_blocks = "warn"
```

[4] The complete list of Clippy lints is available at https://rust-lang.github.io/rust-clippy/master/index.html.

```
multiple_unsafe_ops_per_block = "warn"
unnecessary_safety_doc = "warn"
ptr_as_ptr = "warn"
cast_ptr_alignment = "warn"
transmute_ptr_to_ptr = "warn"
```

This section focuses on Clippy, but `unsafe_op_in_unsafe_fn` is a compiler lint, not a Clippy lint. It's worth mentioning here because it makes unsafe operations explicit, which is the foundation for the Clippy lints that follow. An `unsafe fn` means its caller has extra obligations. It doesn't mean every unsafe operation in the body is automatically justified. With this lint enabled, each dereference, FFI call, or other unsafe operation needs its own `unsafe {}` block. It's allow-by-default in edition 2021 and warn-by-default in edition 2024, so it's worth denying explicitly either way.

Two lints take that idea a step further. `clippy::undocumented_unsafe_blocks` asks for a `// SAFETY:` comment before each unsafe block. `clippy::multiple_unsafe_ops_per_block` encourages each block to contain a single unsafe operation, keeping its safety argument focused. Together they turn a large, vague unsafe region into small claims that can be checked one by one.

`clippy::unnecessary_safety_doc` catches the opposite documentation mistake: a `# Safety` section on a public safe function or trait. A safe API should not have safety preconditions for its callers or implementors to uphold. This is a restriction lint and allow-by-default, so enable it explicitly.

The pointer lints earn their keep when you're porting C habits mechanically. Pointer casts and pointer transmutes often compile without complaint, even when they introduce a stricter alignment requirement or hide a more direct API. A warning doesn't necessarily mean that the code is wrong, but it's a good reason to double-check what the cast is claiming.

When the lint is wrong

Porting from C can also produce the opposite problem. Some C naming conventions conflict directly with rustc's style lints: a mirrored `bm_db_t` typedef triggers `non_camel_case_types`, an extern static trips `non_upper_case_globals`, and a function or variable that keeps its original name trips `non_snake_case`. The group covering all three is `nonstandard_style`.

Sometimes the name genuinely does have to match, and then an `allow` is the right solution. Scope it as tightly as you can:

```
#[allow(non_camel_case_types)]
pub struct bm_db_t {/* … */}
```

A crate-root `#![allow(non_snake_case)]` also silences names you genuinely got wrong. Prefer applying allows to individual items instead of whole modules or entire crates. Generated code you don't control is the exception that can justify a module-wide allow. Put it on the module containing the generated bindings rather than on the entire crate.

You can often avoid an `allow` for functions exposed to C. `no_mangle` uses the Rust function's name for the exported symbol, but `export_name` lets you choose a different name for the C API:

```rust
#[unsafe(export_name = "BMNormalizeURL")]
pub extern "C" fn bm_normalize_url() {
    // ...
}
```

Note: `cheadergen` recognizes `export_name`, so it emits `BMNormalizeURL` in the C header while the Rust function keeps its idiomatic name.

Share lint settings across a workspace

The configuration above applies to one crate. To use it across a workspace, put the tables in the root `Cargo.toml` instead. They've been available since Rust 1.74:

```toml
# Cargo.toml at the workspace root
[workspace.lints.rust]
unsafe_op_in_unsafe_fn = "deny"

[workspace.lints.clippy]
undocumented_unsafe_blocks = "warn"
multiple_unsafe_ops_per_block = "warn"
```

Each member crate then opts in with:

```toml
[lints]
workspace = true
```

`// SAFETY:` comments are part of the code

Consider a pointer that arrives across the FFI boundary:

```rust
use std::ffi::CStr;
let raw = c"https://example.com".as_ptr();
// SAFETY: `raw` is non-null (checked above) and points to a valid
// NUL-terminated byte sequence, as required by `bm_normalize_url`'s contract.
let raw = unsafe { CStr::from_ptr(raw) };
```

The null check happens in this function; the valid NUL-terminated sequence is part of the caller's documented contract. A reviewer can check both claims without re-deriving them from the surrounding code.

Compare that with this:

```rust
use std::ffi::CStr;
let raw = c"https://example.com".as_ptr();
// SAFETY: this is fine
let raw = unsafe { CStr::from_ptr(raw) };
```

That comment carries no information. The next reader still has to establish whether `raw` is non-null, initialized, and terminated. Write the reasoning down.

It's the same discipline as the `# Safety` documentation from Chapter 2. The function's `# Safety` section describes what its caller must guarantee. The local `// SAFETY:` comment describes why one operation can rely on those guarantees at this exact point in the implementation.

Lints are a starting point, not a proof

Run the lints with:

```
cargo clippy --workspace --all-targets
```

`--all-targets` matters here. Without it, `cargo clippy` skips tests, and in these exercises the tests are where most of the FFI calls live.

`cargo clippy --fix` can apply some mechanical fixes. It wants a clean Git tree and refuses to run on a dirty one unless you pass `--allow-dirty`. Review the diff: replacing a pointer cast is easy; deciding whether the pointer is correctly aligned, valid, and live is the more difficult work.

Other mistakes won't trigger a lint at all. No compiler warning can tell you that a C caller lies about a buffer length, or that two libraries expect different allocators. Keep writing tests, use a debugger when they fail, and treat every FFI contract as something to validate at the boundary.

Head to the exercise

You'll work through a small FFI module with the full lint set already switched on. Some warnings are mechanical, others point to a real bug. The point isn't to make `cargo clippy` quiet. It's to understand and document every unsafe operation that remains.

Exercise

The exercise for this section is located in `03_intermediate/04_clippy_lints` \newpage{=latex}

3.5 Out-parameters

A literal port can preserve shapes that exist because of C's type system rather than because of the program's domain. The code may work, but its Rust API still asks callers to follow conventions that Rust can express directly.

The following sections cover five common transformations, starting here with out-parameters. Each one starts with a recognizable C shape, identifies the information hidden in it, and moves that information into a Rust type or an iterator operation.

What an out-parameter is

Some C functions return values by writing them through pointer arguments. These arguments are called out-parameters. For example, this function splits a duration into minutes and seconds:

```
void split_duration(uint32_t total_seconds,
                    uint32_t *out_minutes,
                    uint32_t *out_seconds);
```

`total_seconds` is an input. The function writes its two results through `out_minutes` and `out_seconds`, because a C function can return only one value directly.

Put outputs in the return type

A literal Rust translation preserves those destinations:

```
fn split_duration(total_seconds: u32, out_minutes: &mut u32, out_seconds: &mut u32) {
    *out_minutes = total_seconds / 60;
    *out_seconds = total_seconds % 60;
}
```

The caller must create two variables before it can call the function, even though those variables have no useful value yet. The signature also describes the mechanism inherited from C rather than the values being computed.

Rust can return both values, as a tuple or a struct:

```
fn split_duration(total_seconds: u32) -> (u32, u32) {
    (total_seconds / 60, total_seconds % 60)
}
```

Preserve an existing C boundary

Existing C callers still need the original signature. Keep the out-parameters in a thin compatibility adapter that calls the Rust `split_duration` above and writes its two results through the pointers:

```
fn split_duration(total_seconds: u32) -> (u32, u32) {
    (total_seconds / 60, total_seconds % 60)
}
/// # Safety
///
/// `out_minutes` and `out_seconds` must be valid, writable pointers.
#[unsafe(export_name = "split_duration")]
pub unsafe extern "C" fn split_duration_ffi(
    total_seconds: u32,
    out_minutes: *mut u32,
    out_seconds: *mut u32,
) {
    let (minutes, seconds) = split_duration(total_seconds);
```

```
    // SAFETY: The caller must satisfy the pointer requirements above.
    unsafe {
        out_minutes.write(minutes);
        out_seconds.write(seconds);
    }
}
```

The C ABI remains unchanged, but the out-parameter shape stops at the boundary. The calculation and its Rust callers use an ordinary return value.

Some C APIs allow an output pointer to be `NULL` when the caller doesn't need that output. In that case, the adapter checks each pointer before writing through it.

Recognize values split across channels

An out-parameter can combine with the ordinary return value to describe one logical result. `bm` contains this example:

```
size_t bm_cursor_tags(BmCursor *cursor, const char ***out_tags);
```

The return value is the number of tags, and `out_tags` receives a pointer to the first tag. Together they describe a borrowed sequence. Once the cursor is a safe Rust type, its API can return that sequence as one value:

```
fn tags(&self) -> &[Tag];
```

The same reasoning applies to other paired outputs: identify the value the pieces represent, then choose the Rust return type that represents it directly. The compatibility adapter splits that value back into the return value and out-parameter C expects.

Head to the exercise

In `exercises/03_intermediate/05_out_parameters`, implement `page_location` so that it returns a bookmark's page and slot as a `(page, slot)` tuple instead of writing them through out-parameters. Then complete the C adapter without changing its `void bm_page_location(...)` contract.

Exercise

The exercise for this section is located in `03_intermediate/05_out_parameters` \newpage{=latex}

3.6 Error codes

C APIs commonly report the outcome of an operation with an enum, with one variant for success and one for each kind of failure:

```
typedef enum {
    BM_OK = 0,
    BM_ERR_NOT_FOUND = 1,
    BM_ERR_IO = 2,
    BM_ERR_INVALID_INPUT = 3,
} BmStatus;

BmStatus remove_bookmark(struct Store *store, const char *url);
```

The return type names the codes, but a C enum is still just an integer. C converts it to and from `int` freely and doesn't stop a function from returning a code that makes no sense for it, or one that isn't a variant at all. Correct propagation relies on control flow and review rather than the type system.

Give failures a type

In the safe Rust API, `BM_OK` becomes `Ok(())`, and each failure becomes a variant of an error enum:

```
enum RemoveError {
    NotFound,
    Io,
}

fn remove_bookmark(store: &mut Store, url: &str) -> Result<(), RemoveError>;
```

`BM_ERR_INVALID_INPUT` has no variant. It reports a null pointer or a URL that isn't valid UTF-8, and the safe function can't receive either of those, because the adapter below rejects them first. Don't merge the remaining failures into one generic error because the C version shared a return type.

Map once at the C boundary

An unchanged C ABI still returns its specified numbers. Keep that mapping in one adapter:

```
use std::ffi::{CStr, c_char};

#[derive(Debug)] enum RemoveError { NotFound, Io }
struct Store;
fn remove_bookmark(_: &mut Store, _: &str) -> Result<(), RemoveError> { Ok(()) }
#[repr(C)]
pub enum BmStatus {
    Ok = 0,
    NotFound = 1,
    Io = 2,
    InvalidInput = 3,
}

/// # Safety
```

```
///
/// `store` must point to a valid, uniquely borrowed `Store`. `url` must point
/// to a readable, NUL-terminated byte string.
#[unsafe(export_name = "remove_bookmark")]
pub unsafe extern "C" fn remove_bookmark_ffi(
    store: *mut Store,
    url: *const c_char,
) -> BmStatus {
    let Some(store) = (unsafe { store.as_mut() }) else {
        return BmStatus::InvalidInput;
    };
    if url.is_null() {
        return BmStatus::InvalidInput;
    }

    // SAFETY: The pointer requirements are part of this function's contract,
    // and the null case was rejected above.
    let url = unsafe { CStr::from_ptr(url) };
    let Ok(url) = url.to_str() else {
        return BmStatus::InvalidInput;
    };

    match remove_bookmark(store, url) {
        Ok(()) => BmStatus::Ok,
        Err(RemoveError::NotFound) => BmStatus::NotFound,
        Err(RemoveError::Io) => BmStatus::Io,
    }
}
```

`BmStatus` mirrors the C enum. `#[repr(C)]` gives it the representation the C compiler picks for the same enum, so the two agree on the values they exchange. Returning it to C is sound, because a Rust enum always holds one of its variants. The Rust error enum doesn't need a C representation: its layout stays private because it never crosses the boundary. Conversely, if a raw numeric status arrives from C, match its known values explicitly. Constructing a Rust enum from an arbitrary integer can create an invalid value.

`bm`'s `BmResult` follows this pattern. Its storage layer also shows that not every C failure code is a Rust error: `ENOENT` means a missing file, which may be an empty database, while a read failure or corrupt file remains an error. The domain decides which outcomes are ordinary; the fact that C reported them as integers doesn't.

Head to the exercise

In `exercises/03_intermediate/06_error_codes`, implement `change_bookmark_id` so that it reports its failures as a `Result<(), ChangeIdError>` instead of C status codes, leaving the index unchanged when it fails. Then complete the FFI adapter that maps that result back to the C status codes.

Exercise

The exercise for this section is located in `03_intermediate/06_error_codes` \newpage{=latex}

3.7 Strings

C and Rust use different representations for strings.

A traditional C string is a sequence of bytes followed by a NUL byte (`\0`). A `const char *` usually carries no separate length: functions find the end by scanning for that terminator. The bytes before it aren't necessarily UTF-8.

Rust's `&str` and `String` contain valid UTF-8 and carry their byte length. A `&str` borrows its contents, while a `String` owns them. Neither representation needs a trailing NUL byte.

Rust provides two more types for traditional C strings:

- `CStr` is a borrowed, NUL-terminated sequence of bytes;
- `CString` is an owned, NUL-terminated sequence of bytes.

Passing a string across the boundary therefore takes two steps: establish the C string representation, then decide whether its bytes are valid Rust text.

Borrow a C string as `CStr`

Use `CStr::from_ptr` to borrow a traditional C string:

```
use std::ffi::{CStr, c_char};

/// # Safety
///
/// A non-null `tag` must point to a live, NUL-terminated byte string.
unsafe extern "C" fn tag_byte_len(tag: *const c_char) -> usize {
    if tag.is_null() {
        return 0;
    }

    // SAFETY: The caller supplies a live C string and null was handled above.
    let tag = unsafe { CStr::from_ptr(tag) };
    tag.count_bytes()
}
```

`from_ptr` is `unsafe` because Rust can't determine how much memory is readable or whether a NUL terminator will be found. The resulting `&CStr` borrows the original allocation and can't outlive it.

`count_bytes()` reports the number of bytes before the terminator. Nothing so far has checked or assumed that those bytes are UTF-8. `to_bytes()` exposes those bytes when the API defines the value as arbitrary data. If the value is text, perform the encoding conversion explicitly.

Convert between `CStr` and Rust text

`CStr::to_str` validates UTF-8 and returns a borrowed `&str`, or an error if the bytes aren't valid UTF-8. Turn the `&str` into a `String` when the Rust code needs its own copy.

`to_string_lossy()` replaces invalid sequences, which silently changes the text; use it only when that's the API's stated policy.

Going the other way requires a NUL-terminated representation. `CString::new` creates one from Rust text:

```
use std::ffi::CString;

let tag = String::from("rust");
let c_tag = CString::new(tag)?;
let borrowed = c_tag.as_c_str();
let pointer = borrowed.as_ptr();
let _ = pointer;
Ok::<(), std::ffi::NulError>(())
```

The conversion is fallible because a Rust string may contain an interior NUL. That byte would terminate a C string early. The pointer from `as_ptr()` is valid only while `c_tag` is alive, so don't save a pointer obtained from a temporary `CString`.

Expose a Rust string as a pointer and length

A C API can also describe text using a returned pointer and a length out-parameter:

```
const uint8_t *bm_category_label(uint32_t category, size_t *out_len);
```

A borrowed Rust string already has both pieces. The adapter returns its pointer and writes its byte length through the out-parameter:

```
fn category_label(category: u32) -> &'static str {
    match category {
        1 => "article",
        2 => "reference",
        _ => "other",
    }
}

/// # Safety
///
/// `out_len` must be a valid, writable pointer.
#[unsafe(export_name = "bm_category_label")]
unsafe extern "C" fn category_label_ffi(category: u32, out_len: *mut usize) -> *const u8 {
    let label = category_label(category);

    // SAFETY: The caller must provide a writable output pointer.
    unsafe { out_len.write(label.len()) };
    label.as_ptr()
}
```

The pointer isn't NUL-terminated. C must use the written length instead of `strlen` and must not mutate the bytes. The length is a number of bytes, not Unicode characters.

This example returns a view of a static string, so no allocation or matching destructor is required. When the `&str` borrows from an input or another value instead, C must not retain the returned pointer after that value stops being valid.

Keep the boundary explicit

For each string parameter, specify:

- whether it's a traditional C string or a pointer-and-length view;
- whether `NULL` is accepted and how it differs from an empty string;
- whether the bytes must be UTF-8;
- how long a returned or borrowed pointer remains valid.

Use `CStr` and `CString` for traditional C strings, `&str` and `String` for Rust text, and byte slices when the value isn't text. Convert between them at the boundary instead of treating their pointers as interchangeable.

Head to the exercise

In `exercises/03_intermediate/07_strings`, accept a traditional C string, convert it through `CStr` to `&str`, and return a borrowed tag name as a pointer with its byte length written to an out-parameter. The returned pointer points into the input, so it's valid only while that input string is alive.

Exercise

The exercise for this section is located in `03_intermediate/07_strings` \newpage{=latex}

3.8 Bit flags

C represents a set of independent options by assigning each option a bit:

```
#define FIND_URLS             (1u << 0)
#define FIND_TAGS             (1u << 1)
#define FIND_CASE_INSENSITIVE (1u << 2)

size_t find_bookmarks(const struct Bookmark *bookmarks, size_t len,
                      const char *query, uint32_t flags);
```

Callers combine options with `|` and test them with `&`. The representation is compact and crosses an ABI easily, but `uint32_t` accepts unrelated constants, unknown bits, and combinations the operation may not support.

Name the set of valid bits

The `bitflags` crate generates a transparent newtype, constants, bitwise operators, and set-like methods:

```
use bitflags::bitflags;

bitflags! {
    #[repr(transparent)]
    #[derive(Clone, Copy, Debug, PartialEq, Eq)]
    struct FindFlags: u32 {
        const URLS = 1 << 0;
        const TAGS = 1 << 1;
        const CASE_INSENSITIVE = 1 << 2;
    }
}
```

The storage width matches the C header, and `#[repr(transparent)]` gives the wrapper the layout of that u32. Within Rust, `contains` requires every bit in its argument, while `intersects` asks whether any bit overlaps. `insert` and `remove` mutate a set, and `bits()` returns the raw representation for C.

Named bits don't automatically make every combination meaningful. If a search must inspect at least URLs or tags, validate that separately:

```
use bitflags::bitflags;
bitflags! { struct FindFlags: u32 {
  const URLS = 1; const TAGS = 2; const CASE_INSENSITIVE = 4;
} }
fn validate(flags: FindFlags) -> Result<FindFlags, &'static str> {
    if !flags.intersects(FindFlags::URLS | FindFlags::TAGS) {
        return Err("select URLS, TAGS, or both");
    }
    Ok(flags)
}
```

Choose an unknown-bit policy

Converting a raw C integer requires an explicit compatibility decision:

- `from_bits(raw)` rejects any unknown bit;
- `from_bits_truncate(raw)` discards unknown bits;
- `from_bits_retain(raw)` preserves them.

Rejecting is appropriate when silently ignoring an unknown option would do the wrong thing. Retaining can be useful for a value that must round-trip to a newer library even though this version doesn't interpret every bit. Truncating is a real behavior choice; it isn't a neutral default.

A strict C adapter can therefore convert and validate before entering the safe implementation:

```
use bitflags::bitflags;
bitflags! { #[derive(Clone, Copy)] struct FindFlags: u32 {
  const URLS = 1; const TAGS = 2; const CASE_INSENSITIVE = 4;
} }
fn validate(flags: FindFlags) -> Result<FindFlags, ()> {
  flags.intersects(FindFlags::URLS | FindFlags::TAGS).then_some(flags).ok_or(())
}
fn flags_from_c(raw: u32) -> Result<FindFlags, ()> {
    FindFlags::from_bits(raw).ok_or(()).and_then(validate)
}
```

The safe search function accepts only `FindFlags`, so the boundary is the one place that knows C supplied an untyped integer.

Head to the exercise

In `exercises/03_intermediate/08_bit_flags`, update a set of bookmark output fields using separate enable and disable masks. Detect conflicting changes with typed flags, then strictly convert three raw C masks, apply the update, and return the resulting raw bits.

Exercise

The exercise for this section is located in `03_intermediate/08_bit_flags` \newpage{=latex}

3.9 Loops and iterators

A C loop often does several jobs at once: it traverses memory, controls when iteration stops, and computes a result. A literal Rust port can preserve all of that machinery even after the data has become a safe Rust collection.

Consider this loop from `bm`:

```
for (size_t i = 0; i < b->n_tags; i++) {
    if (strstr(b->tags[i], query))
        return 1;
}
return 0;
```

Initial Rust port

A direct Rust translation looks like this:

```
struct Bookmark { tags: Vec<String> }
fn matches(bookmark: &Bookmark, query: &str) -> bool {
for index in 0..bookmark.tags.len() {
    if bookmark.tags[index].contains(query) {
```

```
        return true;
    }
}
false
}
```

The calculation never uses the numeric value of `index`; it only needs the tag stored there. Rust can iterate over those values directly:

```
struct Bookmark { tags: Vec<String> }
fn matches(bookmark: &Bookmark, query: &str) -> bool {
for tag in &bookmark.tags {
    if tag.contains(query) {
        return true;
    }
}
false
}
```

The index existed only because C needs one to reach each element. A `for` loop over the values is a good final form, especially when its body does several things.

Idiomatic Rust port

The tag loop computes whether any tag matches. The any operation states that directly:

```
struct Bookmark { tags: Vec<String> }
fn matches(bookmark: &Bookmark, query: &str) -> bool {
bookmark.tags.iter().any(|tag| tag.contains(query))
}
```

any() stops after the first match, so the control flow is the same as the early `return` in the C loop. Keep that kind of behavior in mind when you port a loop: short-circuiting, ordering, and whether values are borrowed or moved should all stay the same.

Pointer-bumping loops

The other common C shape walks a string by moving a pointer forward until it reaches the NUL terminator:

```
size_t n = 0;
for (const char *p = s; *p; p++) {
    if (*p == ',')
        n++;
}
```

A literal port keeps the raw pointer, which means every read and every step is `unsafe`, and the loop is only correct as long as the string really ends in a NUL:

```
use std::ffi::c_char;
let s = c"rust,ffi,c".as_ptr();
let mut n = 0;
let mut p: *const c_char = s;
// SAFETY: `s` points to a live, NUL-terminated string, and the loop stops at
// the terminator.
while unsafe { *p } != 0 {
    if unsafe { *p } == b',' as c_char {
        n += 1;
    }
    p = unsafe { p.add(1) };
}
assert_eq!(n, 2);
```

Borrow the C string as a `&CStr` instead. `to_bytes()` gives you the bytes before the terminator as a slice, so there is no terminator to look for and no pointer to move. The loop becomes an iterator over those bytes:

```
let c_str = c"rust,ffi,c";
let n = c_str.to_bytes().iter().filter(|&&byte| byte == b',').count();
assert_eq!(n, 2);
```

Keep raw pointers at the boundary

This refactoring begins after the FFI layer has validated raw pointers and converted them into safe Rust slices, strings, or collections. Pointer validation and slice construction stay in the boundary adapter; the iterator chain operates only on safe Rust values.

Head to the exercise

The starter code in `exercises/03_intermediate/09_loops_and_iterators` is behaviorally correct, so its tests already pass. `wr` fails because Clippy finds index-based loops whose indexes aren't part of the calculation.

Refactor the four functions without changing their results, ordering, short-circuiting, or borrowing behavior. They correspond to any, `position`, `filter().count()`, and a `filter/map/collect` pipeline with a nested any.

Exercise

The exercise for this section is located in `03_intermediate/09_loops_and_iterators`
\newpage{=latex}

3.10 Dynamic analysis with Miri

In Chapter 2 we introduced the "firewall" rules that keep FFI interfaces sane, and in the Clippy section we looked at lint rules that help enforce them. Lints and compiler settings can't observe

behavior, though. They can only read your code as the compiler does, and infer from that.

We'll now introduce another class of tooling to complement static analysis: dynamic analysis with Miri.

Miri is an interpreter for rustc's MIR (mid-level intermediate representation). It runs your code, checking each operation against the Rust Abstract Machine. Miri is especially useful for `unsafe` Rust, since it catches incorrect initialization, use-after-free, and borrowing-rule violations: a good fit for the kind of systems code we tend to translate from C to Rust. Do note that Miri requires a nightly compiler. This repository's `rust-toolchain.toml` already pins one with the `miri` component included, so the exercises work without any extra setup.

What Miri catches

When interpreting your Rust code, Miri tracks the initialization state and provenance of each allocation. It will flag reads outside the bounds of your allocation, before it's correctly initialized (unless you use `MaybeUninit`), or after it's freed. Miri also catches data races, including weak-memory reorderings, and the aliasing violations that complex unsafe code tends to produce, such as writing through a mutable reference while other code still holds references to the same allocation. It also catches memory leaks: if a test ends without freeing every allocation, for example through `Box::into_raw` or `CString::into_raw` without a matching `from_raw`, Miri will fail the test.

The downsides

Miri is by design an interpreter, meaning it can only ever see code paths that actually execute. Code paths never exercised by your test suite are invisible to Miri. This also means non-deterministic bugs need luck: a bug that only happens on a specific thread interleaving will remain flaky.[5] Miri is also slow: expect a 3,000–7,000× slowdown in some cases.

Quick Miri cheat sheet

Using Miri is just like using Cargo, but with `miri` inserted into the commands. For example, to run unit tests: `cargo miri test -p <crate>`. You can also run binaries, examples, etc.

To pass flags to Miri itself, use the `MIRIFLAGS` environment variable.

The first `cargo miri` invocation builds a Miri-specific sysroot, which takes a minute or two. Run `cargo miri setup` once ahead of time to get that out of the way.

Quite often you will need to tweak your code and test cases to run under Miri at reasonable speed, or at all. For example, you may want to reduce test iterations when running under Miri or disable certain incompatible tests. For this, the `miri` cfg attribute exists. You can write `#[cfg(miri)]` on expressions to enable them *only* when executing under Miri. For incompatible tests you can write `#[cfg_attr(miri, ignore)]`.

[5] The `-Zmiri-many-seeds` option runs several executions at once, which helps catch flakes more often. Miri also has the `-Zmiri-address-reuse-rate`, `-Zmiri-address-reuse-cross-thread-rate`, `-Zmiri-preemption-rate`, and `-Zmiri-compare-exchange-weak-failure-rate` options that help with catching threading bugs more often. But generally it's better to turn to tools like `loom` for proper concurrency testing.

Note that Miri (being an interpreter) also accepts any `--target` triple which is handy to test certain target features such as big-endian or 32-bit without running an actual emulator of some sort.

Miri and C

Miri is a pure Rust interpreter and cannot run native libraries. The Miri team hand-wrote shims for common ones such as libc, which are built into Miri, but for the long tail of native libraries there is no support. Unfortunately, our mixed C-Rust codebase sits right in that long tail. So the question isn't "can Miri check my mixed codebase?" (it cannot), but "how much of it can I get Miri to check?"

The strategy we most often employ is "thin glue, thick core": pure-Rust core with `unsafe extern "C"` functions (the uncheckable part) being extremely simple adapters only. This means the vast majority of the code we actually care about is already Miri-checkable, and FFI functions can be validated using lints, reviews, and other tools if necessary.

The only complicating factor is Rust calling into C: our Rust code calling a not-yet-translated module, for example, or some native library we do not intend to port at all. For this we use hand-written shims (just like the Miri team did) that swap out the C functions for Rust stubs when running under Miri. Keep in mind that the stub doesn't need to implement the C code at all; it simply needs to return the right values: error codes, out-params, and buffers of a given length. One neat side effect is that "stubbing out a C function" forces you to confront the details of a function that we gloss over when we simply call it. That usually leads to a much better understanding of the code, which makes it a worthwhile exercise in itself.

You would usually have something like this:

```
#[cfg(miri)]
mod stubs {
    #[unsafe(no_mangle)]
    unsafe extern "C" fn SomeCFunction() {
        // ... some stub here ...
    }
}
```

In a regular build we would link against the `.so` library and get the `SomeCFunction` symbol that way. Under Miri we stub out the symbol, pointing it at a simple pure-Rust function. The stubs live in their own module because Rust won't let a definition and the `extern` declaration of `SomeCFunction` share a name.

One word of caution, though: Miri passing with Rust stubs does not mean your codebase with the full C functions is correct. These stubs, written correctly, can give you confidence that the final product will also be correct, but they are not a substitute for full-program tests. It is easy to accidentally test your stubs' behavior more than the actual C code's.

Future features

Miri has gained support for running code directly from native libraries: `MIRIFLAGS="-Zmiri-native-lib=path/to/libbm.so"` loads the .so and directly calls the symbols in it. This is a

promising direction, but for now it is limited. It does not support Windows, supports only integer and pointer arguments and returns (no structs by value), and by design cannot check most Rust invariants: memory shared with C stops being tracked for init and provenance. On Linux there is even more experimental support for native tracing that attempts to trace which bytes C touched and keep tracking for all others, but none of it is ready for production use yet.

Head to the exercise

The exercise is in `exercises/03_intermediate/10_miri`. It contains a small Rust wrapper around `tag_normalize` from `tag.c`, a module we haven't ported yet. Its tests pass with a plain `cargo test` but fail under Miri.

Exercise

The exercise for this section is located in `03_intermediate/10_miri` \newpage{=latex}

3.11 Performance benchmarking

You have ported a C function to Rust. The tests pass and the output is identical. Great! But what about execution speed?

It is tempting to run the program once, look at the wall clock, and call that a benchmark. The problem is that one number tells you almost nothing. Your laptop may boost one run and throttle the next. The filesystem cache may already be warm, and your browser may wake up and steal a core.

A *benchmark* repeatedly measures a controlled workload. It can report *latency*, how long one operation takes, or *throughput*, how many operations finish in a given time. Our benchmark will measure the latency of one call to `bm_normalize_url`.

A *profile* answers a different question: where did a program spend CPU time, allocate memory, or perform I/O? We focus on benchmarking here because it tells us whether the C-to-Rust port regressed. Profiling appears briefly at the end, as a way to investigate an unexpected result.

Benchmarking tools

The main tool for running benchmarks is `cargo bench`. By itself it does nothing and requires a benchmark harness. At the time of writing, Rust's built-in benchmark harness is nightly-only: `#[bench]`, the `test` crate, and `test::Bencher` are all unstable.[6] In stable Rust, you need a third-party harness such as Criterion or Divan. Criterion stores baselines and reports changes between runs. Divan provides a smaller, attribute-based harness and reports the fastest, slowest, median, and mean times. For simplicity, we will use Divan.

Add Divan to the crate that owns the Rust port. The exercise also uses the `cc` crate to compile the C baseline:

[6] See the `test` library feature in the Unstable Book for the current status of `#[bench]` and `test::Bencher`.

```
# Cargo.toml
[build-dependencies]
cc.workspace = true

[dev-dependencies]
divan = "0.1"

[[bench]]
name = "normalize"
harness = false
```

`harness = false` tells Cargo not to start its built-in test harness. The benchmark binary supplies its own `main` function and starts Divan instead:

```
fn main() {
    divan::main();
}
```

The build script compiles the original C implementation:

```
// Sample build.rs
fn main() {
    cc::Build::new()
        .file("c_src/normalize.c")
        .compile("bm_normalize_c");
}
```

This keeps both implementations in the same Cargo build. When `cargo bench` selects the optimized bench profile, `cc` reads Cargo's `OPT_LEVEL` and applies the corresponding optimization level to the C code too.[7]

If you build the C code manually instead of using `cc`, pass the appropriate `-O` flag to the C compiler yourself. Otherwise you may end up comparing optimized Rust with unoptimized C, which makes the benchmark meaningless.

Benchmark the same work

Now we need to compare the C implementation with the Rust port. A C/Rust comparison is only useful when both benchmarks include the same work.

Start from the C signature. The caller owns the output buffer, and nothing is allocated:

```
BmResult bm_normalize_url(const char *raw, char *out, size_t out_len);
```

Porting it gives you a choice, and this decides what you are able to measure:

[7] The `cc` crate reads the optimization level from Cargo's `OPT_LEVEL` environment variable. See the `cc::Build::opt_level` documentation.

```rust
// Safe and idiomatic. Not comparable: it allocates, and reaching it from
// C's `const char *` costs a UTF-8 check on the way in.
pub fn normalize_url(raw: &str) -> Result<String, InvalidUrl>;

// Comparable: the same contract as C, so the two benchmarks do the same work.
pub unsafe extern "C" fn bm_normalize_url_rust(
    raw: Option<NonNull<c_char>>,
    out: Option<NonNull<c_char>>,
    out_len: usize,
) -> BmResult;
```

If you benchmark the first one against C, the Rust number carries a validation and an allocation that the C number does not: you have measured two different APIs, not two implementations of the same operation.

This does not mean the safe API is wrong. The allocation may well be the right design. It does mean you need to state what the benchmark includes.

For a direct port comparison, take the second signature, and give both implementations the same input and equivalent reusable output buffers:

```rust
use std::ffi::c_char;
type BmResult = i32;
unsafe extern "C" {
    #[link_name = "bm_normalize_url"] // bind the C symbol under a second name
    fn bm_normalize_url_c(raw: *const c_char, out: *mut c_char, out_len: usize) -> BmResult;
}

#[divan::bench(sample_size = 4096)]
fn c(bencher: divan::Bencher) {
    let raw = b"HTTPS://EXAMPLE.COM/Rust\0";
    let mut out = [0 as c_char; 2048]; // created once, so it is not timed

    bencher.bench_local(|| unsafe {
        let result = bm_normalize_url_c(
            divan::black_box(raw.as_ptr().cast()),
            divan::black_box(out.as_mut_ptr()),
            out.len(),
        );
        divan::black_box(result)
    });
}
```

The benchmark setup happens before `bench_local`, so creating the output buffer is not timed. The closure mutably captures that buffer and reuses it for every iteration. `bench_local` runs single-threaded.

These calls take only a few dozen nanoseconds. `sample_size = 4096` runs each benchmark that many times in every timing sample; Divan then divides the total time by 4096. This keeps each sample well above timer precision and makes the result less sensitive to an interruption during Divan's automatic calibration. Use the same sample size for both implementations; adjust it for slower or faster workloads.

> **`black_box` is not decorative.** Without it, the compiler can notice that the result goes nowhere and optimize away some or all of the work. You would then be wowed by the speed of a function that did not run, making the whole benchmark meaningless.
>
> **Rule of thumb:** Benchmark the semantics you want to compare, not the convenience API you happened to write first.

Check correctness first, then run both benchmarks, from the workspace root:

```
cargo test -p bm_benchmarking
cargo bench -p bm_benchmarking --bench normalize
```

You can pass a filter to Divan when you only want one implementation:

```
cargo bench -p bm_benchmarking --bench normalize -- rust
```

Divan invokes each function many times. It groups those invocations into samples and reports a per-iteration time:

- **median** is the middle sample and usually the most useful value for a quick comparison;
- **mean** is the average and moves more when one sample is unusually slow;
- **fastest** and **slowest** show the extremes and help expose noisy runs;
- **samples** and **iters** show how many measurements and function calls contributed to the result.

The fastest-to-slowest range shows only the extremes within one run. A single outlier can make it huge, so do not use it to decide whether two implementations differ.

Check for a regression

Suppose repeated runs put Rust around 220 ns and C around 200 ns. That is a 10% difference.

It is a *performance regression* only if the slowdown remains under the same workload and conditions. Run the benchmark several times on the same otherwise-idle machine and compare the medians. If the difference keeps changing direction, it is too small to distinguish from noise under those conditions. If Rust remains slower by roughly the same amount, investigate.

The instructions are deterministic; the conditions under which they run are not. Caches, frequency scaling, and whatever else the machine is doing all land in your measurement.

Treat a repeatable regression like any other bug:

1. Run the correctness tests first. A faster wrong port is not an optimization.
2. Record the benchmark command, input, machine, and baseline median.
3. Change one thing, then rerun the same benchmark under the same conditions.
4. If the regression remains, profile the slow path before changing more code.

Keep the benchmark after the fix. It is now the thing that will catch the regression if it comes back.

> **CI tip:** A continuous-benchmarking tool such as Bencher can store results over time and fail a pull request when performance regresses.

A brief note on profiling

If a benchmark finds a repeatable regression, a profiler can show where the extra work happens.

`cargo-samply` builds an optimized binary with debug information and records CPU samples. See its documentation for installation and usage. Look for functions that occupy the widest frames in the profile. Those are where the sampled CPU time accumulated. Then record a profile for the C implementation, or for the version before your change, and compare it with the new profile. This shows which functions now take more CPU time and explains the slower benchmark.

Hotpath is useful when you want timing or allocation measurements for a small set of functions in an application. Its instrumentation is disabled unless you enable the corresponding Cargo features:

```
# Cargo.toml
[dependencies]
hotpath = "0.24"

[features]
hotpath = ["hotpath/hotpath"]
hotpath-alloc = ["hotpath/hotpath-alloc"]
```

Mark the functions you want to measure and the application's entry point:

```
#[hotpath::measure] // time every call to this function
fn normalize_url(raw: &str) -> String {
    // ...
}

#[hotpath::main] // install the reporter, print the table on exit
fn main() {
    // ...
}
```

```
cargo run --release --features='hotpath,hotpath-alloc'
```

Hotpath can also write JSON reports for CI. Its `hotpath-utils` command compares reports from the pull request and the base branch, including per-function timing and allocation differences.[8] This is useful for watching a few representative application paths. Keep Divan for controlled microbenchmarks, and disable Hotpath while collecting the final Divan numbers because instrumentation changes the code being measured.

Head to the exercise

The exercise is in `exercises/03_intermediate/11_benchmarking`. It ships with the C benchmark. Replace the `bm_normalize_url_rust` stub and add the matching Rust benchmark, then run both and compare their median latencies.

The parity test compares return codes on every input, but compares the output buffers only when the call succeeded. The C version lowercases into the caller's buffer before it looks for a tab,

[8] The Hotpath CI guide shows how to produce the head and base JSON reports and compare them in GitHub Actions.

so it writes into `out` even on inputs it rejects. Your port validates first, so it doesn't. That is a difference differential testing is supposed to surface, and one worth keeping: the test pins it down rather than asking you to reproduce it.

Exercise

The exercise for this section is located in `03_intermediate/11_benchmarking` \newpage{=latex}



mainmatter.com/rust
info@mainmatter.com